Lithium Diffusion in a Spherical Particle: Effect of Flux Shape, Frequency and Diffusivity

Ishitva Yadav
CL 443 — Supervisor: Prof. Jason Picardo

October 30, 2026

1 Problem

Lithium diffusion inside a spherical particle is studied using a PDE model. The concentration inside the particle varies with radius and time, and lithium enters or leaves only through the surface. The outputs looked at are the surface concentration C_s, the average concentration C_{avg}, and the concentration profile $C(r)$ inside the particle. Different flux inputs and different diffusivity values are applied to the same particle to see how they change the response. Six experiments are reported. Experiments 1 to 4 change one thing at a time (24 PDE simulations). Experiments 5 and 6 build two-equation models for C_s and C_{avg}, by the Galerkin method and by SINDy, and test them against further PDE simulations.

2 Model

The model is Fick's second law in spherical coordinates, written in dimensionless form. The scalings used are

$$\tau = \frac{D_0 t}{R^2}, \qquad r = \frac{\tilde{r}}{R}, \qquad C = \frac{c}{c_0}, \qquad f(C) = \frac{D(c)}{D_0}, \qquad \delta = \frac{jR}{D_0 c_0}, \tag{1}$$

so τ is scaled time, r runs from 0 at the centre to 1 at the surface, C is concentration as a fraction of full, $f(C)$ is the diffusivity relative to the reference value D_0, and δ is the scaled surface flux. The governing equation is

$$\frac{\partial C}{\partial \tau} = \frac{1}{r^2}\frac{\partial}{\partial r}\left[r^2 f(C)\frac{\partial C}{\partial r}\right], \tag{2}$$

with boundary and initial conditions

$$f(C)\left.\frac{\partial C}{\partial r}\right|_{r=1} = -\delta(\tau), \tag{3}$$

$$\left.\frac{\partial C}{\partial r}\right|_{r=0} = 0, \tag{4}$$

$$C(r,0) = 1. \tag{5}$$

The two reported quantities are

$$C_s(\tau) = C(1,\tau), \qquad C_{\text{avg}}(\tau) = 3\int_0^1 C(r,\tau)\, r^2\, dr. \tag{6}$$

Integrating Equation (2) over the particle and using the boundary conditions gives

$$\frac{dC_{\text{avg}}}{d\tau} = -3\,\delta(\tau), \tag{7}$$

which ties the average concentration directly to the applied flux.

2.1 Parameters

The particle radius is $R = 2 \times 10^{-6}$ m, the maximum concentration is $c_0 = 51554$ mol m^{-3}, and the reference diffusivity is $D_0 = 2 \times 10^{-14}$ m^2 s^{-1}. These give $R^2/D_0 = 200$ s, so one unit of τ is 200 seconds. The particle starts full and uniform, $C_0 = 1$, and each run of Experiments 1 to 4 covers $0 \le \tau \le 10$, that is 2000 seconds. The reference surface flux is $j_0 = 4.3 \times 10^{-6}$ mol m^{-2} s^{-1}, which gives $\delta_0 = j_0 R/(D_0 c_0) = 0.008341$. A C-rate multiplier of 3 is used, so the flux amplitude in every deterministic profile is

$$A = 3\,\delta_0 = 0.025022. \tag{8}$$

The equations are solved in MATLAB using `pdepe` in spherical mode on a grid of 1000 radial points and 2000 time points, with relative and absolute tolerances of 10^{-9} and a maximum step of 10^{-2} in τ. These settings are the same for all 24 runs of Experiments 1 to 4. Experiments 5 and 6 use their own run lengths and time steps, given there.

2.2 Diffusivity

Unless stated otherwise the diffusivity is the fitted NMC811 curve used in the solver:

$$f(C) = p_4 + (p_1 - p_4)\exp\left[-\exp\left(-p_2(C - p_3)\right)\right], \tag{9}$$

with $p_1 = 0.1008$, $p_2 = -8.2105$, $p_3 = 0.6986$ and $p_4 = 0.0009$. The NMC811 diffusivity is concentration dependent and is used as the default material model. It is low near $C = 1$, about 0.0009, and rises to about 0.1 at low C. Experiment 3 replaces it with a constant f.

2.3 Flux profiles

Six deterministic flux shapes are used:

$$\texttt{const}: \quad \delta(\tau) = A, \tag{10}$$

$$\texttt{stepUp}: \quad \delta(\tau) = A\left[1 + \tfrac{1}{2}\tanh 20(\tau - 5)\right], \tag{11}$$

$$\texttt{linear}: \quad \delta(\tau) = A\,\frac{\tau}{10}, \tag{12}$$

$$\texttt{linearDown}: \quad \delta(\tau) = A\left(1 - \frac{\tau}{10}\right), \tag{13}$$

$$\texttt{sin}: \quad \delta(\tau) = A\sin(\omega\tau), \tag{14}$$

$$\texttt{cos}: \quad \delta(\tau) = A\cos(\omega\tau). \tag{15}$$

The frequency is $\omega = 20$ everywhere except in Experiment 2, where it is swept. Since `sin` and `cos` are negative for half of each cycle, they push lithium back into the particle, so C can rise above 1 in those runs. The model puts no upper bound on C.

3 Experiment 1: flux shape

- The six flux profiles were applied to the same particle with the same diffusivity model.
- Only the shape of $\delta(\tau)$ was changed; $\omega = 20$ for `sin` and `cos`.

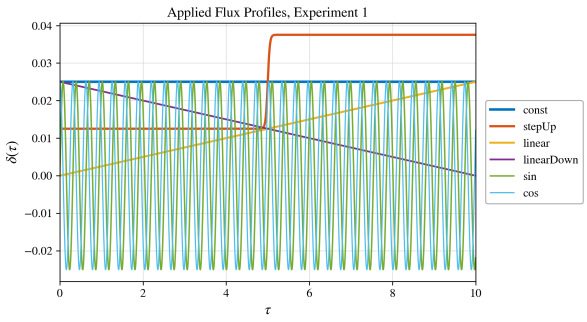


Figure 1: The six applied flux profiles $\delta(\tau)$.

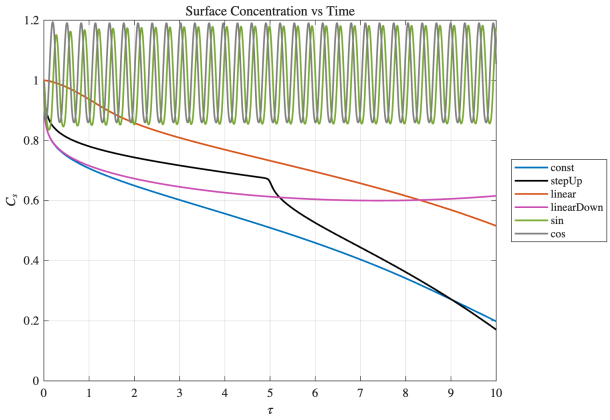


Figure 2: Surface concentration C_s against τ.

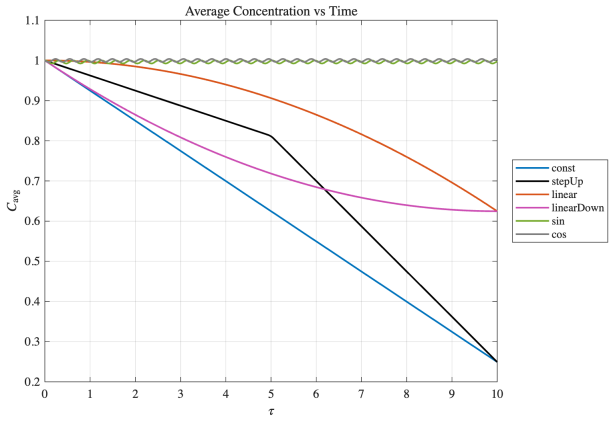


Figure 3: Average concentration C_{avg} against τ.

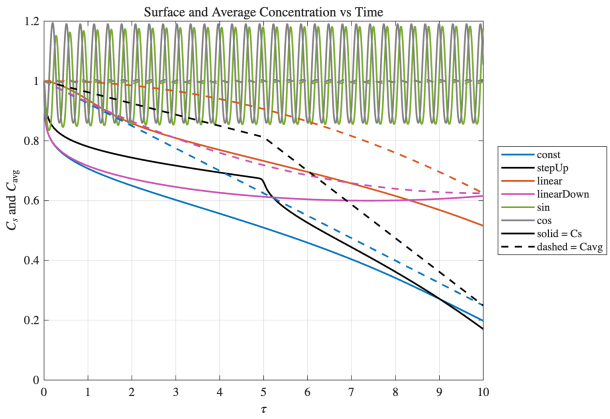


Figure 4: C_s and C_{avg} against τ.

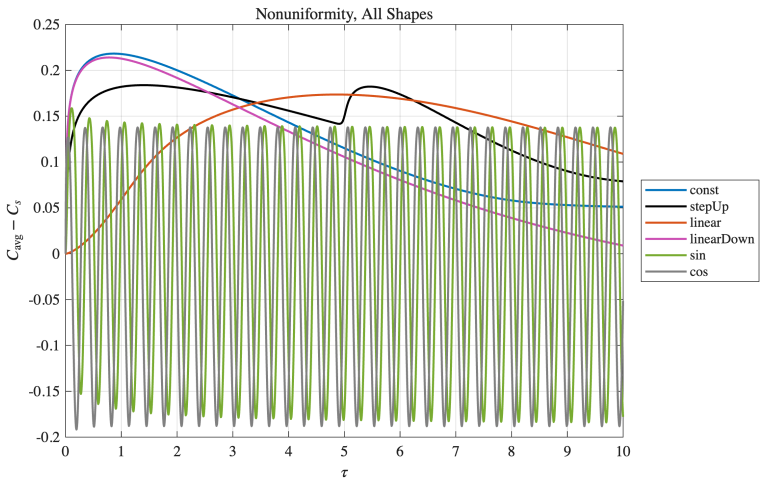


Figure 5: $C_{\text{avg}} - C_s$ against τ.

- C_s changes with the flux shape, while C_{avg} follows the total charge applied.

- `const` and `stepUp`: same charge 0.7507, same final $C_{\text{avg}} = 0.2493$, but C_s minima of 0.1981 and 0.1704.
- `linear` and `linearDown`: same charge 0.3753, same final $C_{\text{avg}} = 0.6247$, but C_s minima of 0.5158 and 0.5999.
- This follows from Equation (7): C_{avg} depends only on the integral of δ.
- Profiles that load the surface harder or earlier give a larger gap. `linearDown` peaks at $\tau = 0$, where the diffusivity is lowest, and reaches a maximum gap of 0.2140 against 0.1736 for `const`.

3.1 Inside the particle

- $r = 0$ is the centre and $r = 1$ is the surface.
- A flat curve means the particle is uniform; a drop near $r = 1$ means the surface is emptied faster than diffusion refills it.

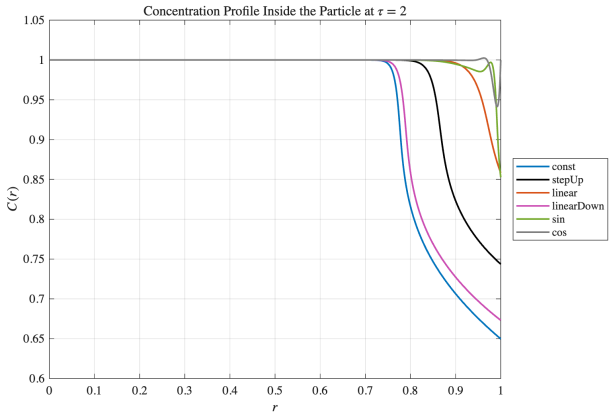


Figure 6: Concentration profile at $\tau = 2$.

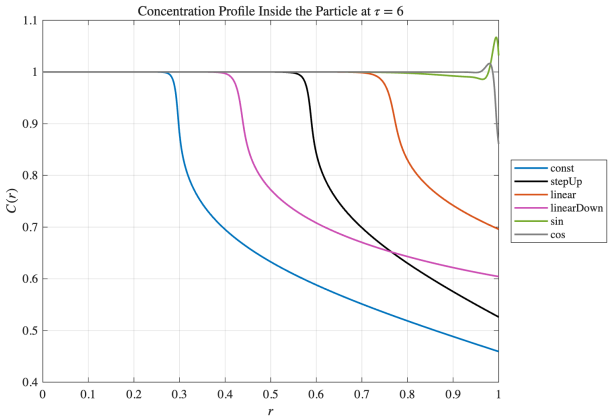


Figure 7: Concentration profile at $\tau = 6$.

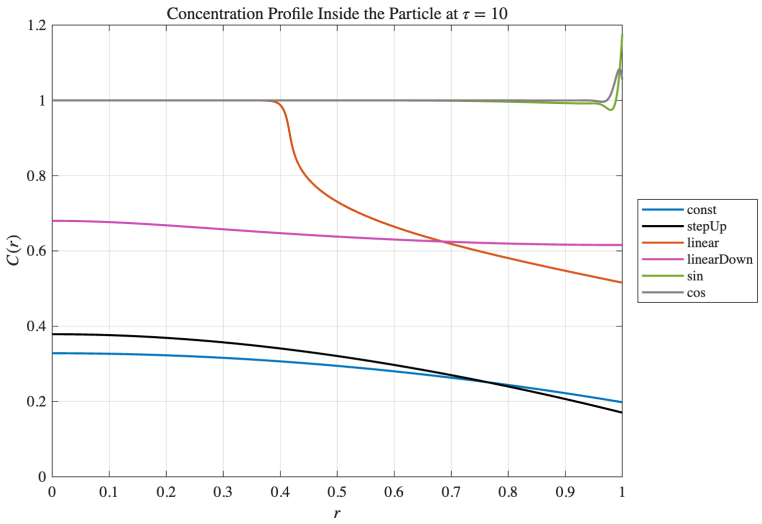


Figure 8: Concentration profile at $\tau = 10$.

- The steady and ramped profiles change the concentration across the whole particle.
- `sin` and `cos` disturb only a thin layer near the surface and leave the interior close to its initial value.

4 Experiment 2: frequency of the oscillating flux

- The flux was fixed at $\delta(\tau) = A\sin(\omega\tau)$ and only ω was changed, taking 2, 5, 10, 20, 40 and 80.
- The material model and all other solver settings were kept fixed.
- The period is $T_\tau = 2\pi/\omega$, which gives 628.3, 251.3, 125.7, 62.8, 31.4 and 15.7 seconds.
- The time plots use the window $5 \leq \tau \leq 6$ (200 seconds), because at $\omega = 80$ the full run holds more than 120 oscillations. Line thickness decreases as ω increases. The full solution is kept in the saved data.

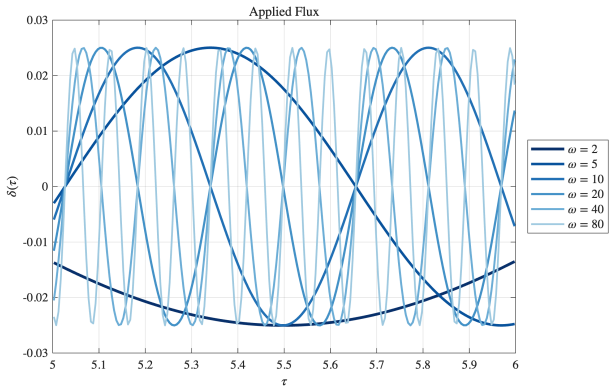


Figure 9: Applied flux $\delta(\tau)$ for the six frequencies.

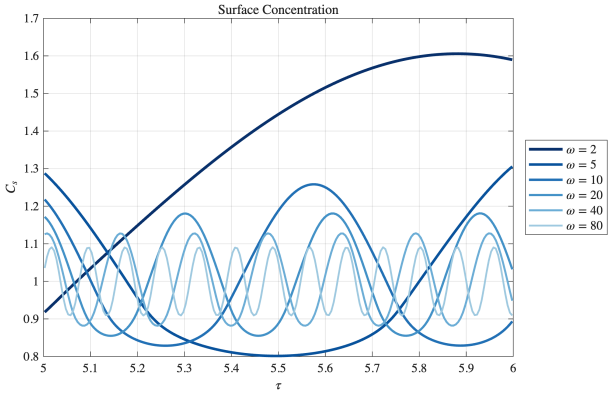


Figure 10: Surface concentration C_s against τ.

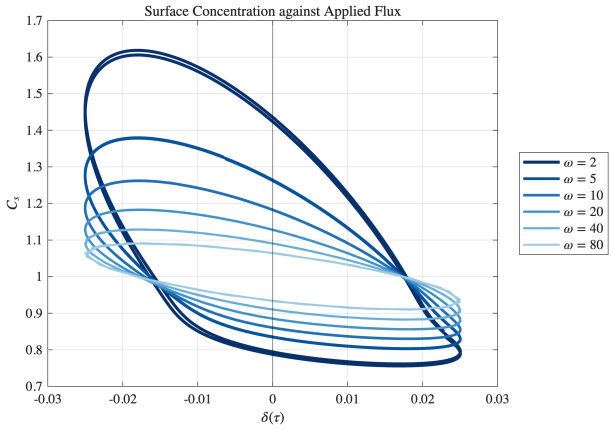


Figure 11: Surface concentration C_s against applied flux δ.

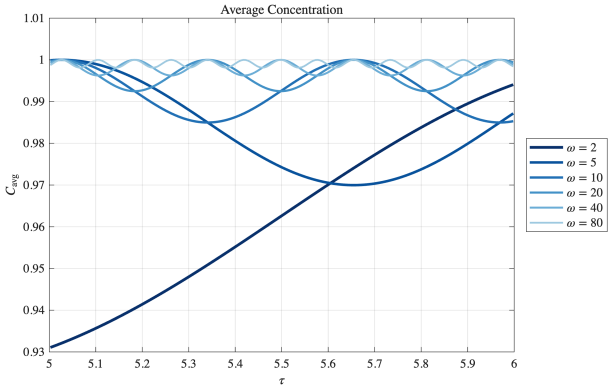


Figure 12: Average concentration C_{avg} against τ.

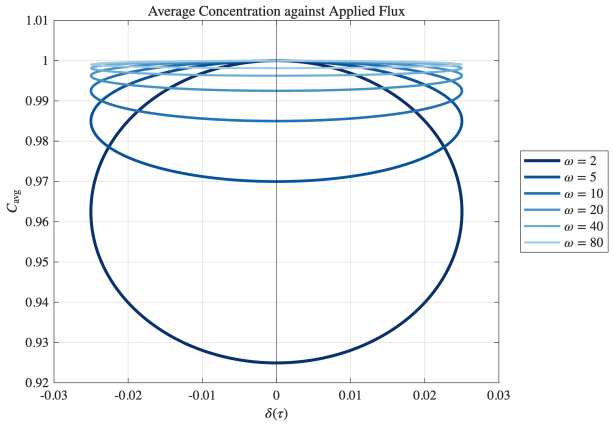


Figure 13: Average concentration C_{avg} against applied flux δ.

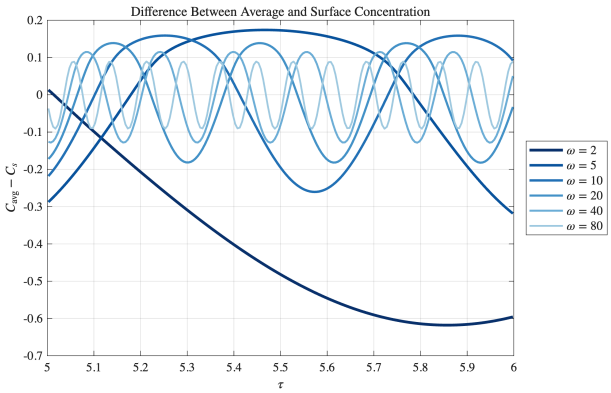


Figure 14: $C_{\text{avg}} - C_s$ against τ.

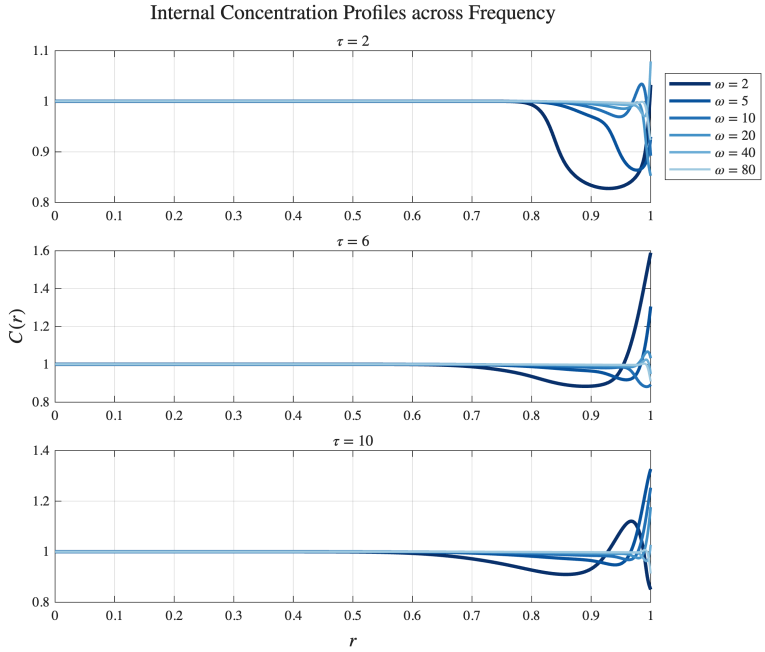


Figure 15: Concentration profiles at $\tau = 2$, 6 and 10.

- Increasing ω reduces the visible oscillation in C_s: it spans 0.729 to 1.618 at $\omega = 2$ and 0.889 to 1.092 at $\omega = 80$, with the intermediate cases in order.
- C_{avg} changes very little. Its minimum rises from 0.925 to 0.998, because a faster oscillation reverses sign before much charge has accumulated.
- C_s against δ is a loop, not a single line, so C_s is not set by the instantaneous flux alone. The loops shrink as ω increases.
- The disturbed region moves closer to the surface: the profile is curved from about $r = 0.55$ at $\omega = 2$, and flat until about $r = 0.97$ at $\omega = 80$.
- The maximum gap $C_{\text{avg}} - C_s$ falls from 0.215 to 0.110.

5 Experiment 3: constant diffusivity

- Equation (9) was replaced by a constant, so $f(C) = f$ at every concentration, taking 0.01, 0.1, 1, 10 and 100.
- The flux was fixed at $\delta(\tau) = A\sin(20\tau)$ and all other settings were kept fixed.
- D_0 was not changed, so all five runs share the same time axis. The same window $5 \leq \tau \leq 6$ is used.

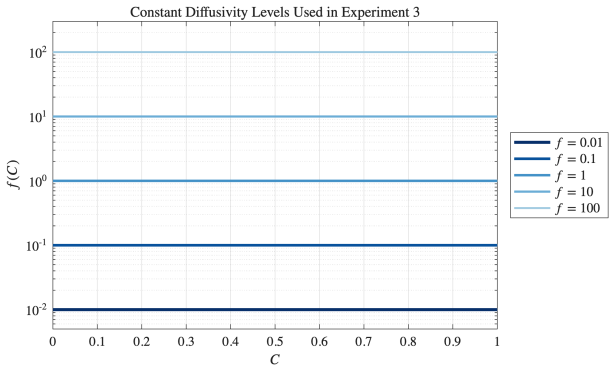


Figure 16: The five constant diffusivity levels (log vertical axis).

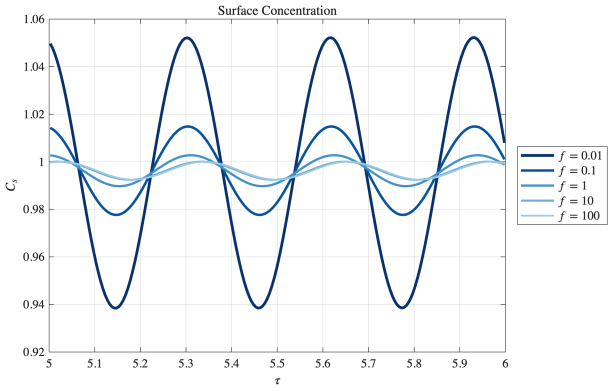


Figure 17: Surface concentration C_s against τ.

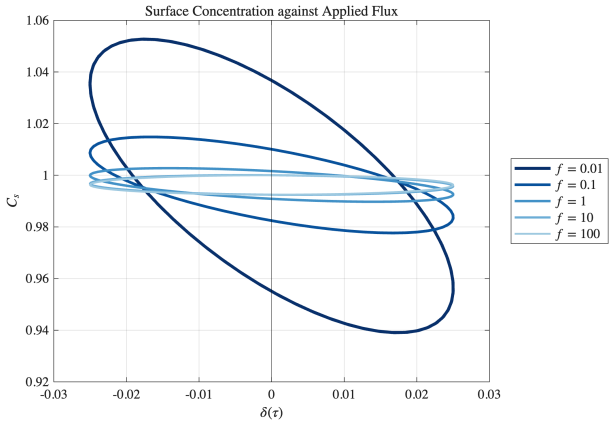


Figure 18: Surface concentration C_s against applied flux δ.

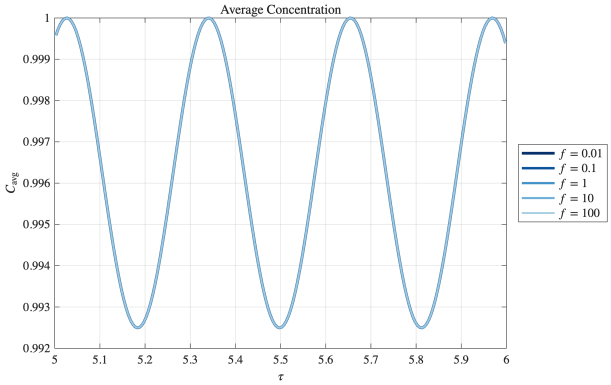


Figure 19: Average concentration C_{avg} against τ.

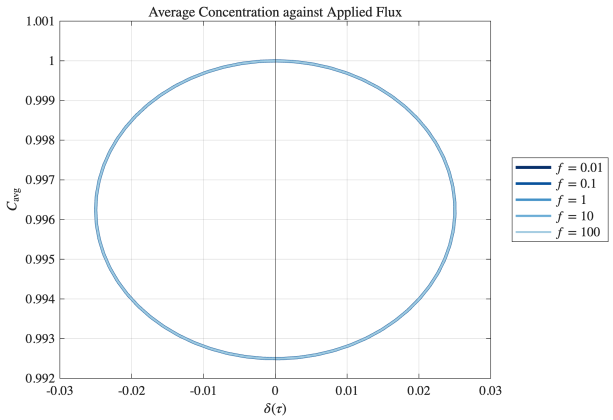


Figure 20: Average concentration C_{avg} against applied flux δ.

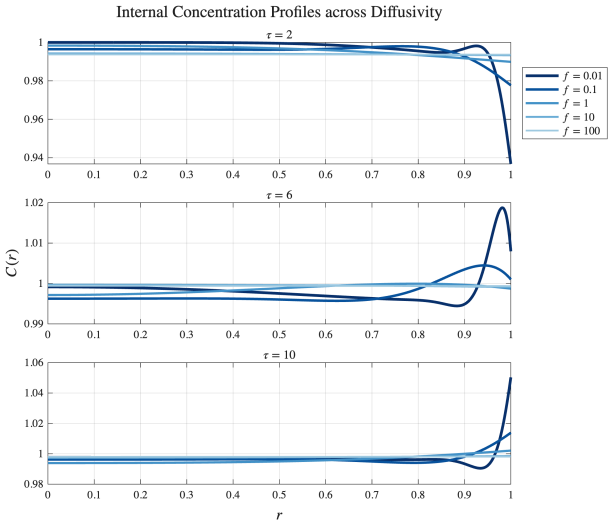


Figure 21: Concentration profiles at $\tau = 2$, 6 and 10.

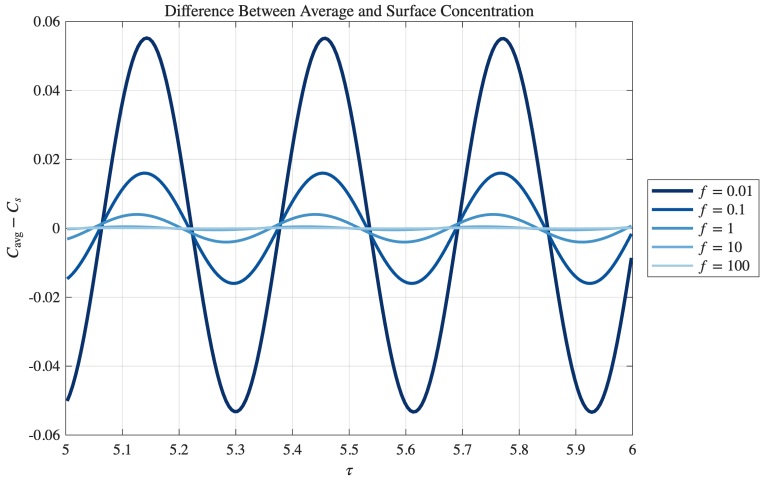


Figure 22: $C_{\text{avg}} - C_s$ against τ.

- At low diffusivity the profile is strongly non-uniform near the surface. As f increases the gradient becomes smaller and the profile approaches a flat line.
- At $f = 0.01$ the profile is flat through the interior and turns sharply in the last few percent of the radius, reaching 1.05 at the surface at $\tau = 10$ while the centre sits near 0.996. At $f = 100$ it is flat across the whole radius.
- The C_s range narrows with f: 0.9228–1.0527, then 0.9740–1.0148, 0.9897–1.0028, 0.9924–1.0001 and 0.99249–1.00000.
- The maximum gap $C_{\text{avg}} - C_s$ falls from 0.0710 to 0.0198, 0.00418, 0.000499 and 0.000051.
- The C_{avg} histories are essentially the same in all five runs, spanning 0.99249 to 1.00000 with a final value of 0.99808, because the applied flux is the same and Equation (7) contains no diffusivity.

6 Experiment 4: randomly generated flux

- `trueRandom` generates a reproducible piecewise-constant flux from a fixed seed. It divides the run into 17 segments of random length and gives each segment a random amplitude.
- Changing the seed gives a different excitation while the generation method and all solver settings stay unchanged. This is useful for generating varied input–output data for later modelling.
- Five seeds were used: 30303, the original seed in the project and the reference case, then 101, 202, 303 and 404.
- Only the flux and the surface concentration are plotted.
- The amplitude convention differs from the deterministic profiles: an internal constant of 0.01 is used instead of $A = 0.025022$, and the mean is subtracted from the amplitude sequence. The net charge removed is -0.024 to $+0.018$, against 0.751 for a steady discharge at $\delta = A$, so these are excitation signals rather than discharges and should not be compared directly with Experiment 1.

6.1 Individual seeds

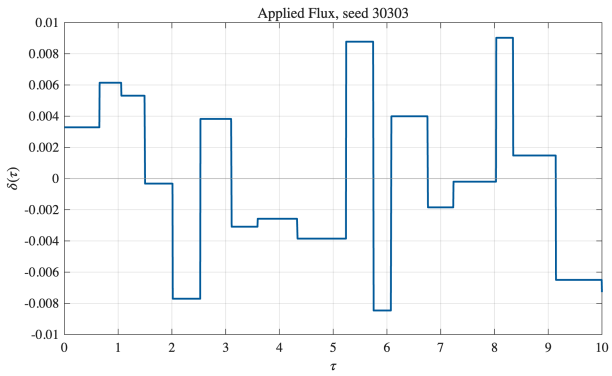


Figure 23: Seed 30303 (reference). Applied flux $\delta(\tau)$.

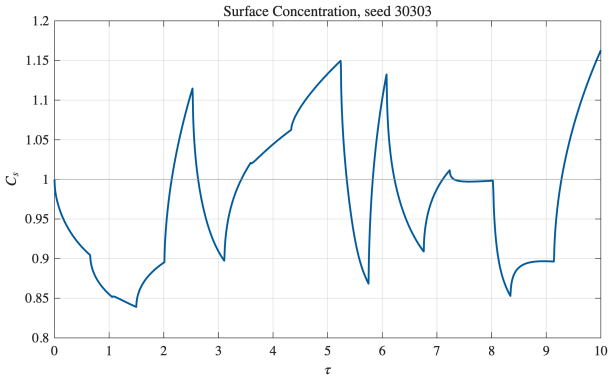


Figure 24: Seed 30303. Surface concentration C_s.

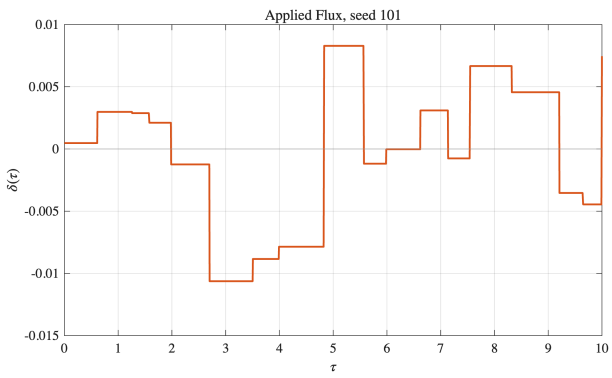


Figure 25: Seed 101. Applied flux $\delta(\tau)$.

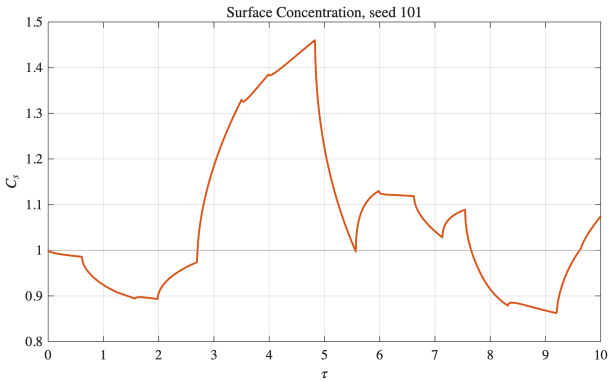


Figure 26: Seed 101. Surface concentration C_s.

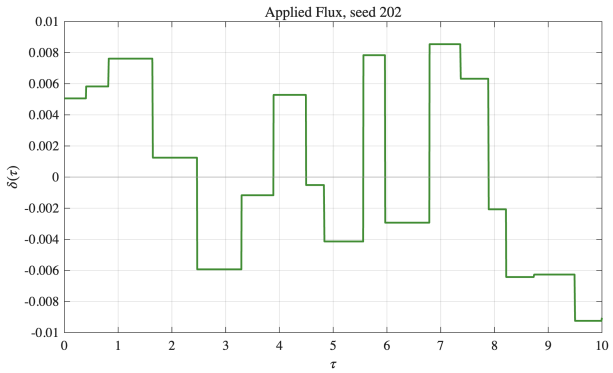


Figure 27: Seed 202. Applied flux $\delta(\tau)$.

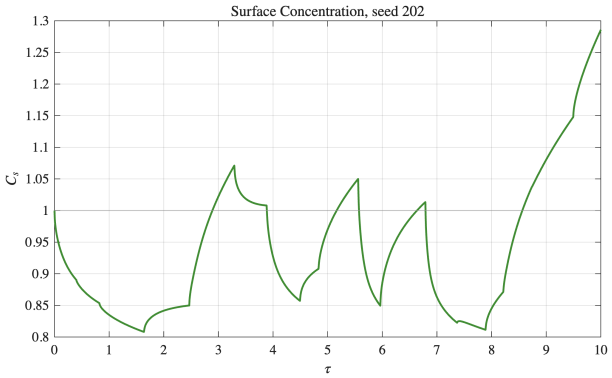


Figure 28: Seed 202. Surface concentration C_s.

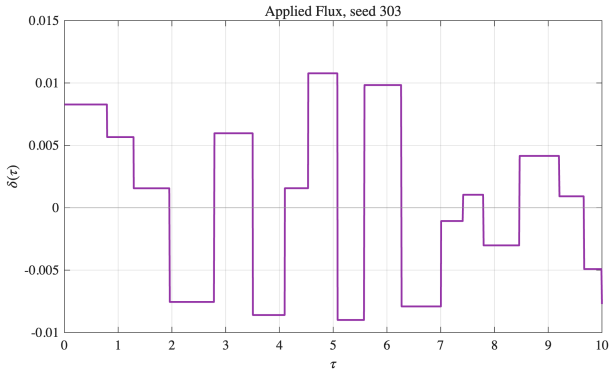


Figure 29: Seed 303. Applied flux $\delta(\tau)$.

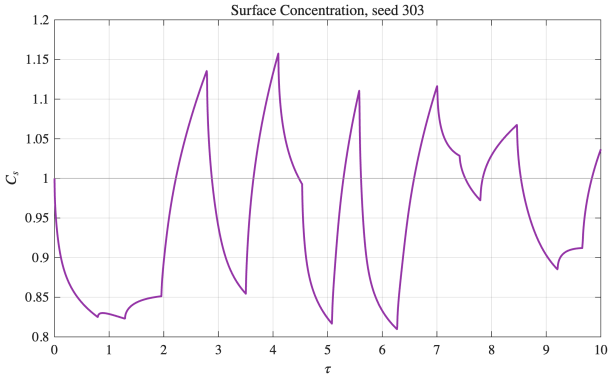


Figure 30: Seed 303. Surface concentration C_s.

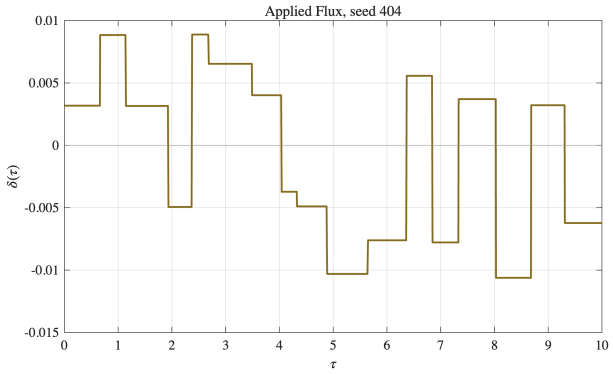


Figure 31: Seed 404. Applied flux $\delta(\tau)$.

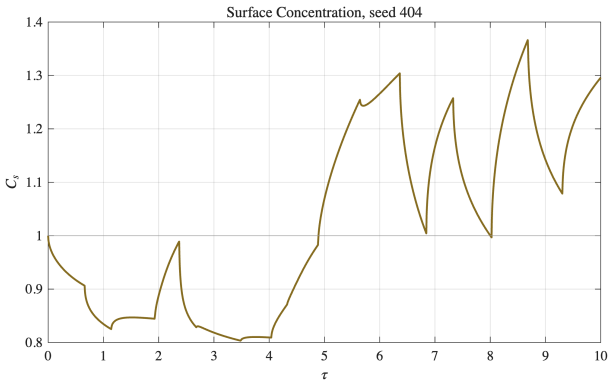


Figure 32: Seed 404. Surface concentration C_s.

6.2 All seeds together

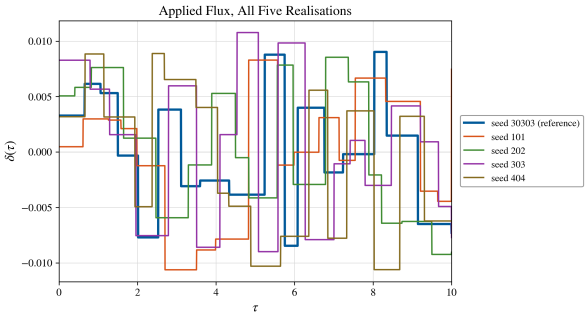


Figure 33: Applied flux $\delta(\tau)$ for all five seeds.

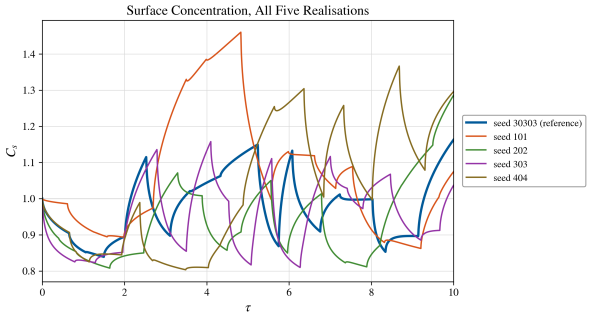


Figure 34: Surface concentration C_s for all five seeds.

- The five fluxes differ from each other, with segment edges at different times and different sign patterns.
- The surface responses are similar in character: irregular excursions above and below $C_s = 1$ with no steady drift.
- Across the five seeds C_s spans 0.803 to 1.460 and C_{avg} spans 0.949 to 1.049.
- The minimum of C_s varies only between 0.803 and 0.862, while the maximum varies more widely, between 1.158 and 1.460.
- Five seeds only show that the behaviour is similar in character, not anything about its distribution. The per-seed values are saved in `Results/tables/trueRandom_summary.csv`.

7 Experiment 5: two-equation model by the Galerkin method

- The aim is a model of two ordinary differential equations, one for C_{avg} and one for C_s, derived from the PDE by the Galerkin method, and then tested against the PDE.
- The diffusivity is constant, $f = 1$, throughout this experiment and Experiment 6.

7.1 Starting point: the Galerkin formulas

- The concentration is approximated by N fixed radial shapes $P_k(r)$, each multiplied by a coefficient that depends only on time:

$$C(r,\tau) \approx \sum_{k=1}^{N} \alpha_k(\tau)\, P_k(r). \tag{16}$$

- The shapes are orthogonal with the spherical weight r^2, and the first shape is $P_1 = 1$:

$$\int_0^1 r^2\, P_i(r)\, P_j(r)\, dr = 0 \quad (i \neq j), \qquad P_1(r) = 1. \tag{17}$$

- The Galerkin projection of the PDE onto these shapes gives the following two results, in dimensionless form. They are the starting point of this experiment.

$$\frac{dC_s}{d\tau} = -\delta(\tau) \sum_{i=1}^{N} \frac{P_i(1)^2}{\int_0^1 r^2 P_i(r)^2\, dr} - \sum_{i=1}^{N} \frac{P_i(1) \int_0^1 r^2 P_i'(r)\, f\left(\sum_{k=1}^N \alpha_k P_k(r)\right) \left(\sum_{k=1}^N \alpha_k P_k'(r)\right) dr}{\int_0^1 r^2 P_i(r)^2\, dr}, \tag{18}$$

$$\frac{dC_{\text{avg}}}{d\tau} = \frac{d\alpha_1}{d\tau} = \frac{-\delta(\tau)}{\int_0^1 r^2\, dr} = -3\,\delta(\tau). \tag{19}$$

- $f(\cdot)$ is the diffusivity evaluated at the approximate concentration. With constant f, it is taken out of the integral in Equation (18).
- Equation (19) is the exact mass balance, Equation (7).
- Equation (18) gives $dC_s/d\tau$ in terms of the coefficients α_k. To obtain a model in C_s and C_{avg}, the shapes are chosen and the α_k are written in terms of C_s and C_{avg}.

7.2 Choice of shapes, $N = 2$

- $P_1 = 1$. The second shape is taken as $P_2 = r^2 + c$, the simplest shape that is even in r (so $\partial C/\partial r = 0$ at the centre) and can bend.
- The constant c is fixed by the orthogonality condition, Equation (17), between P_1 and P_2:

$$\int_0^1 r^2 \cdot 1 \cdot (r^2 + c)\, dr = \frac{1}{5} + \frac{c}{3} = 0 \quad \Rightarrow \quad c = -\frac{3}{5}, \qquad P_1(r) = 1, \qquad P_2(r) = r^2 - \frac{3}{5}. \tag{20}$$

- The approximation is $C = \alpha_1 + \alpha_2 \left(r^2 - \frac{3}{5}\right)$, a parabola in r.

7.3 Linking α_1, α_2 to C_{avg} and C_s

Average concentration. Using the definition $C_{\text{avg}} = 3\int_0^1 C(r,\tau)\, r^2\, dr$ and the approximation:

$$C_{\text{avg}} = 3\left[\alpha_1 \int_0^1 r^2\, dr + \alpha_2 \int_0^1 r^2 \left(r^2 - \tfrac{3}{5}\right) dr\right] = 3\left[\frac{\alpha_1}{3} + \alpha_2 \cdot 0\right] = \alpha_1. \tag{21}$$

- The second integral is zero by the orthogonality condition, since $\int_0^1 r^2 P_2\,dr = \int_0^1 r^2 P_1 P_2\,dr = 0$. So the first coefficient is exactly the average concentration, $\alpha_1 = C_{\text{avg}}$, which is why Equation (19) is written as $dC_{\text{avg}}/d\tau$.

Surface concentration. At $r = 1$:

$$C_s = \alpha_1 P_1(1) + \alpha_2 P_2(1) = \alpha_1 + \frac{2}{5}\,\alpha_2. \tag{22}$$

- With $\alpha_1 = C_{\text{avg}}$:

$$\alpha_1 = C_{\text{avg}}, \qquad \alpha_2 = \frac{5}{2}\,(C_s - C_{\text{avg}})\,. \tag{23}$$

7.4 Applying the formula for $dC_s/d\tau$

	$P_i(1)$	$\int_0^1 r^2 P_i^2\,dr$	$P_i'(r)$
$P_1 = 1$	1	1/3	0
$P_2 = r^2 - 3/5$	2/5	4/175	$2r$

Table 1: Values needed in Equation (18).

- $\int_0^1 r^2 P_2^2\,dr = \int_0^1 \left(r^6 - \frac{6}{5}r^4 + \frac{9}{25}r^2\right) dr = \frac{1}{7} - \frac{6}{25} + \frac{3}{25} = \frac{4}{175}$.
- **First sum (flux term):**

$$\frac{P_1(1)^2}{1/3} + \frac{P_2(1)^2}{4/175} = \frac{1}{1/3} + \frac{4/25}{4/175} = 3 + 7 = 10.$$

- **Second sum (diffusion term).** Since $P_1' = 0$, only $i = 2$ contributes, and inside the bracket only $k = 2$ contributes: $\sum_k \alpha_k P_k'(r) = 2r\,\alpha_2$. With constant f:

$$\frac{P_2(1)\,f \int_0^1 r^2\,(2r)(2r\,\alpha_2)\,dr}{4/175} = \frac{\frac{2}{5}\cdot\frac{4}{5}\,f\,\alpha_2}{4/175} = 14\,f\,\alpha_2.$$

- So Equation (18) becomes

$$\frac{dC_s}{d\tau} = -10\,\delta - 14\,f\,\alpha_2.$$

- Replace α_2 using Equation (23): $14f\alpha_2 = 14f \cdot \frac{5}{2}(C_s - C_{\text{avg}}) = 35f(C_s - C_{\text{avg}})$.

7.5 The two-equation Galerkin model

$$\boxed{\frac{dC_{\text{avg}}}{d\tau} = -3\,\delta(\tau), \qquad \frac{dC_s}{d\tau} = -35f\,(C_s - C_{\text{avg}}) - 10\,\delta(\tau), \qquad C_s(0) = C_{\text{avg}}(0) = 1.} \tag{24}$$

- With $f = 1$, written term by term: $dC_s/d\tau = -35\,C_s + 35\,C_{\text{avg}} - 10\,\delta$. These three coefficients are compared with SINDy in Experiment 6.
- The initial values follow from $C(r, 0) = 1$: $\alpha_1 = 1$, $\alpha_2 = 0$.
- Assumptions: constant f; the profile is a combination of 1 and r^2 at every instant; the particle starts uniform.
- Slow-flux check: when δ changes slowly, C_s and C_{avg} move together, so $dC_s/d\tau \approx -3\delta$, which gives $C_s - C_{\text{avg}} = -\delta/(5f)$.
- With $N = 2$ the equation for C_s contains only C_s, C_{avg} and δ. With $N \geq 3$ it would also contain $\alpha_3, \ldots$, which cannot be written with C_s and C_{avg} alone.

7.6 Test against the PDE

- The flux is $\delta(\tau) = A\cos(\omega\tau)$ at $\omega = 0.5$, 1, 2, 3, 5, 10, 20, 30, 50, 100 and 200.
- Each run lasts $\tau_{\max} = \max(10,\ 5 \text{ periods})$ with time step $\min(0.005,\ \text{period}/100)$, on the same 1000-point radial grid.
- The Galerkin model, Equation (24), is solved on the same time points, starting from $C_s = C_{\text{avg}} = 1$.

Two questions had to be settled before the comparison could be read: how to measure the error, and over which part of each run.

7.6.1 Why a relative error

The first idea was to look at the plain (absolute) error in C_s. Over the last period of each run it rises from about 4×10^{-7} at $\omega = 0.5$ to about 8×10^{-4} at $\omega = 200$, roughly 2000 times larger. On its own this number says little, because the size of the signal changes too: the swing of C_s (its maximum minus its minimum, the *band* in which the values lie) shrinks from 0.30 to 0.0037, about 80 times smaller.

ω	band of $C_{s,\text{PDE}}$ (last period)	RMS absolute error (last period)	Rel (%)
0.5	0.3006	3.99×10^{-7}	0.0001
1	0.1507	1.60×10^{-6}	0.0011
2	0.0763	6.37×10^{-6}	0.0084
3	0.0518	1.42×10^{-5}	0.0275
5	0.0329	3.85×10^{-5}	0.1172
10	0.0196	1.38×10^{-4}	0.7031
20	0.0130	3.90×10^{-4}	2.9984
30	0.0104	5.95×10^{-4}	5.7293
50	0.0078	7.95×10^{-4}	10.1721
100	0.0054	8.69×10^{-4}	16.1875
200	0.0037	7.79×10^{-4}	20.9499

Table 2: Absolute and relative error of the Galerkin model over the last period.

So at high frequency the error grows while the band it must be compared with shrinks. The relative error divides the error by that band. For a predicted signal $\hat{y}$ and a reference y, sampled at the n points $t_1, \ldots, t_n$ of a time *window* W,

$$\text{Rel}_W(\hat{y}, y) = \frac{\text{RMS}_W(\hat{y} - y)}{\max_W y - \min_W y}, \qquad \text{RMS}_W(e) = \sqrt{\frac{1}{n}\sum_{k=1}^{n} e(t_k)^2}, \tag{25}$$

given in % (100×Rel). Here $\hat{y} = C_{s,\text{model}}$ and $y = C_{s,\text{PDE}}$:

$$\text{Rel (\%)} = 100 \times \frac{\text{RMS}\left(C_{s,\text{model}} - C_{s,\text{PDE}}\right)}{\max C_{s,\text{PDE}} - \min C_{s,\text{PDE}}}.$$

The numerator and the denominator always use the *same window*, stated with every result.

Why: it measures the error against how much C_s actually moves, so it punishes both effects at once: the growing error and the shrinking band. Between $\omega = 0.5$ and 200 it rises about 1.6×10^5 times, which shows how quickly the model loses accuracy. Dividing by C_s itself was rejected: C_s stays close to 1, so every error would look tiny.

Decided: this formula is used for every error in this report, in Experiment 6 as well.

7.6.2 Why both the whole run and the last period

- **Whole run.** Every run starts from a uniform particle, and the flux is switched on at full size at $\tau = 0$. In a real battery the same happens whenever the current changes suddenly: the start of charging or discharging, or a step in current. Diffusion cannot keep up, a steep layer forms near the surface, and the model must follow this transient. The whole-run error includes it. At low frequency the largest whole-run error (about 9.5×10^{-4}) occurs right at the start, near $\tau = 0.005$.
- **Last period.** After the start-up the response settles into a repeating pattern. The last-period error measures the model in this settled state.

Why: the two windows answer two different questions: how well the model follows a sudden change, and how well it follows a steady operation. Comparing them shows how much of the error comes from the start-up. At $\omega = 0.5$ the whole-run error is 40 times the last-period error; above $\omega = 20$ the two are almost equal, because there the model is inaccurate in every period.

7.6.3 Why one period is enough

The first version of this test used the last two periods. In the meeting with the professor it was pointed out that with constant f and a sine or cosine flux the problem is linear, so once the start-up has died out each period is a copy of the one before. This was checked:

ω	$\max \lvert C_s(\tau) - C_s(\tau - P) \rvert$ over the last period	Rel %, last 2 periods	Rel %, last period
0.5	1.2×10^{-7}	0.0001	0.0001
3	4.3×10^{-9}	0.0275	0.0275
30	7.5×10^{-8}	5.717	5.729
200	8.9×10^{-10}	20.996	20.950

Table 3: Periodicity of the PDE solution, and the Galerkin error over one and two periods.

Consecutive periods agree to within 10^{-7}, and the error is the same over one or two periods (the small differences come only from where the sample points fall). One period is used from here on.

7.6.4 Result

	C_s range		$\max \lvert C_s - C_{s,\mathrm{PDE}} \rvert$		Rel (%)	
ω	PDE	Galerkin	whole run	last period	whole run	last period
0.5	0.84971–1.15029	0.84971–1.15029	9.50×10^{-4}	5.75×10^{-7}	0.005	0.0001
1	0.92463–1.07537	0.92463–1.07537	9.50×10^{-4}	2.29×10^{-6}	0.015	0.0011
2	0.96186–1.03814	0.96185–1.03815	9.50×10^{-4}	9.02×10^{-6}	0.043	0.0084
3	0.97408–1.02592	0.97407–1.02593	9.50×10^{-4}	2.01×10^{-5}	0.081	0.0275
5	0.98357–1.01643	0.98354–1.01646	9.50×10^{-4}	5.45×10^{-5}	0.172	0.1172
10	0.99022–1.00978	0.99003–1.00997	9.49×10^{-4}	1.95×10^{-4}	0.735	0.7031
20	0.99349–1.00652	0.99299–1.00701	9.40×10^{-4}	5.51×10^{-4}	2.998	2.9984
30	0.99481–1.00524	0.99425–1.00577	9.35×10^{-4}	8.37×10^{-4}	5.672	5.7293
50	0.99609–1.00398	0.99581–1.00427	1.13×10^{-3}	1.13×10^{-3}	10.109	10.1721
100	0.99732–1.00275	0.99763–1.00249	1.26×10^{-3}	1.23×10^{-3}	16.046	16.1875
200	0.99814–1.00191	0.99877–1.00130	1.13×10^{-3}	1.11×10^{-3}	20.752	20.9499

Table 4: Galerkin model against the PDE, cosine flux, $f = 1$.

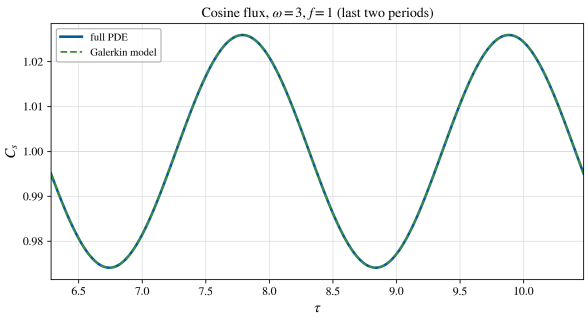


Figure 35: C_s, PDE and Galerkin model, $\omega = 3$.

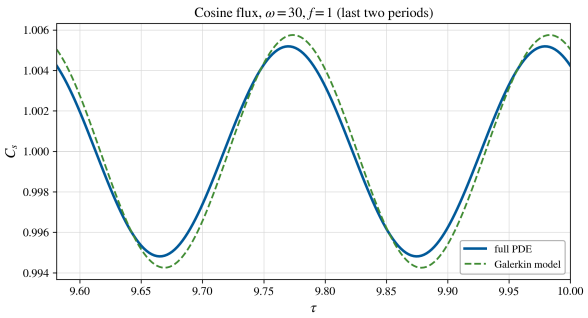


Figure 36: C_s, PDE and Galerkin model, $\omega = 30$.

- At low frequency the Galerkin model and the PDE are almost the same: below $\omega = 5$ the error over the last period is under 0.12%.
- The error grows with frequency, reaching about 21% at $\omega = 200$. A fast flux changes the concentration only in a thin layer near the surface, and a parabola across the whole particle cannot represent that.
- At low frequency the largest error over the whole run occurs at the very start, near $\tau = 0.005$: the cosine flux switches on at full size at $\tau = 0$ and the parabola cannot follow that sudden start.

8 Experiment 6: SINDy

- SINDy (sparse identification of nonlinear dynamics, Brunton, Proctor and Kutz, 2016) finds an equation directly from data. The time derivative of a quantity is written as a combination of candidate terms (the library), and only the terms that are needed are kept.
- The aim is to find the equation for $dC_s/d\tau$ from PDE data and compare its coefficients with the Galerkin model of Experiment 5.
- Before using it on the PDE, the SINDy code and the method of choosing its settings are checked on the Lorenz system, whose equations are known.

8.1 Method

- Data are collected at many times and stacked: $\dot{X} = \Theta(X)\,\Xi$, where each column of Θ is one candidate term evaluated on the data, and Ξ holds the unknown coefficients.
- Ξ is found by sequentially thresholded least squares (STLSQ): fit by least squares, set every coefficient smaller than a threshold λ to zero, fit again using only the remaining terms, and repeat (10 times). The threshold is applied to the coefficients directly.

8.1.1 Training, validation and test data

- **Training data**: the data SINDy is fitted on. E_{train} is measured on it.
- **Validation data**: new runs that were not used in the fit. They are used to score every model in the λ sweep and to choose λ. E_{dv} and E_{sim} are measured on them.
- **Test data**: a third set, used *once*, after λ has been chosen. It is not used for any choice, so it gives an honest score of the chosen model.

Decided: one unseen test set for each problem, used once, after training and validation are complete.

8.1.2 The three model errors

All three use Equation (25). When there are several variables (for Lorenz x, y, z), Rel is computed for each variable separately, with its own band, and the results are averaged, so every variable counts equally whatever its size.

- E_{train}: Rel of $\Theta(X)\Xi$ against the derivative it was fitted to, on the training data. It can only go down as more terms are kept, so it cannot tell a real term from an unnecessary one.
- E_{dv} (derivative on validation data): Rel of $\Theta(X)\Xi$ against the derivative of new data, with Θ evaluated at the *true* states of the new data. It asks: given the true state, does the model predict the right rate of change?
- E_{sim} (simulation on validation data): the SINDy equations are *solved* from the starting point of the new data, and the solution is compared with the true states. It asks: if the model is used on its own, does it follow the true trajectory?

They are written out in full for Lorenz in Section 8.2.3 and for the PDE in Section 8.3.3.

8.1.3 The λ grid

λ is swept on a log scale with **four values per decade**, $\lambda_k = 10^{k/4}$. Each step multiplies or divides λ by $10^{0.25} \approx 1.78$:

$$\ldots,\ 0.178,\ 0.316,\ 0.562,\ 1.0,\ 1.78,\ 3.16,\ 5.62,\ 10,\ \ldots$$

So after $\lambda = 1$ the next smaller value is 0.562, not 0.99 or 0.9. This is why the selected Lorenz λ is "0.56". The model itself changes just below $\lambda = 1$, because the smallest true Lorenz coefficients

are ±1; a finer grid would put the edge of the plateau closer to 1 but would select the same model. **Decided:** keep four values per decade.

8.2 Check on the Lorenz system

Why: before trusting SINDy on the PDE, where the right answer is not known, it is tested on a system whose equations are known exactly. If it recovers them, the code and the method of choosing its settings can be trusted.

8.2.1 System, data and derivatives

$$\dot{x} = \sigma(y - x), \qquad \dot{y} = x(\rho - z) - y, \qquad \dot{z} = xy - \beta z, \qquad \sigma = 10,\ \rho = 28,\ \beta = 8/3, \tag{26}$$

starting from $x(0) = -8$, $y(0) = 7$, $z(0) = 27$. With these values the true equations are $\dot{x} = -10x + 10y$, $\dot{y} = 28x - y - xz$, $\dot{z} = -2.6667z + xy$: seven terms in total. SINDy has to find these from data alone.

- Training data every $\Delta t = 0.001$ up to $t = 100$ (10^5 points). Library: all polynomials in x, y, z up to fifth order (56 terms, from 1 to z^5). 10 threshold-and-refit passes.

SINDy needs the derivatives $\dot{X}$. Three ways of getting them are compared:

$$\text{A (exact):} \quad \dot{X} \text{ from the equations,}$$

$$\text{C2 (2nd-order central):} \quad \dot{X}_i \approx \frac{X_{i+1} - X_{i-1}}{2\Delta t}, \qquad \text{error} \propto \Delta t^2,$$

$$\text{C4 (4th-order central):} \quad \dot{X}_i \approx \frac{-X_{i+2} + 8X_{i+1} - 8X_{i-1} + X_{i-2}}{12\,\Delta t}, \qquad \text{error} \propto \Delta t^4.$$

For C2 and C4 the one or two points at each end, where the central formula cannot be used, are left out.

Why: A is the ideal reference. C2 is what is used on the PDE, where only data are available. C4 was added on the professor's suggestion, to see whether a more accurate derivative changes the result. An earlier case, "exact derivative plus noise", was removed: the added term was a systematic change to the data rather than measurement noise, and it did not belong in this check.

Equation	Term	True	A (exact)	C2 (2nd order)	C4 (4th order)
$\dot{x}$	x	-10.0000	-10.000000	-9.999780	-10.000000
$\dot{x}$	y	10.0000	10.000000	9.999779	10.000000
$\dot{y}$	x	28.0000	28.000000	27.997984	28.000000
$\dot{y}$	y	-1.0000	-1.000000	-0.999606	-1.000000
$\dot{y}$	xz	-1.0000	-1.000000	-0.999944	-1.000000
$\dot{z}$	z	-2.6667	-2.666667	-2.666589	-2.666667
$\dot{z}$	xy	1.0000	1.000000	0.999971	1.000000
$\max \lvert\xi_{\text{SINDy}} - \xi_{\text{true}}\rvert$			5.2×10^{-14}	2.0×10^{-3}	2.3×10^{-7}

Table 5: Lorenz coefficients of the selected model (the 7-term model, chosen in Section 8.2.7). All three cases find exactly the 7 true terms; all other 161 coefficients are zero. The last row is the maximum coefficient error (Section 8.2.3). The 4th-order differences are about 10^4 times more accurate than the 2nd-order ones.

8.2.2 Training, validation and test runs

- **Training:** the run above, from $(-8, 7, 27)$, $0 \le t \le 100$.
- **Validation** (to score every model and choose λ): three new runs from $(5, -5, 20)$, $(-3, 4, 30)$ and $(8, 8, 15)$, $0 \le t \le 20$, $\Delta t = 0.001$.
- **Test** (used once, after λ is chosen): three further runs from $(1, -1, 25)$, $(-6, 6, 18)$ and $(10, 3, 32)$, $0 \le t \le 20$, $\Delta t = 0.001$.

Decided: the Lorenz test set is three more new trajectories.

8.2.3 How the error of a model is measured

Below, $\dot{x}_{\text{ref}}$ is the derivative of the data, taken by the *same* method as in training (A, C2 or C4), and $\dot{x}_{\text{SINDy}} = \Theta(x, y, z)\,\xi_x$ is the SINDy equation for $\dot{x}$, evaluated at the true x, y, z of the data; likewise for y and z. The errors are given as fractions.

Training error. On the training run:

$$E_{train} = \frac{1}{3}\left[\frac{\text{RMS}(\dot{x}_{\text{SINDy}} - \dot{x}_{\text{ref}})}{\max \dot{x}_{\text{ref}} - \min \dot{x}_{\text{ref}}} + \frac{\text{RMS}(\dot{y}_{\text{SINDy}} - \dot{y}_{\text{ref}})}{\max \dot{y}_{\text{ref}} - \min \dot{y}_{\text{ref}}} + \frac{\text{RMS}(\dot{z}_{\text{SINDy}} - \dot{z}_{\text{ref}})}{\max \dot{z}_{\text{ref}} - \min \dot{z}_{\text{ref}}}\right].$$

Derivative error on validation data. The same formula on each validation run $j = 1, 2, 3$ (window: the whole run, $0 \le t \le 20$), then the mean over the three runs:

$$E_{dv} = \frac{1}{3}\sum_{j=1}^{3} \frac{1}{3} \sum_{v \in \{x,y,z\}} \frac{\text{RMS}\left(\dot{v}_{\text{SINDy}} - \dot{v}_{\text{ref}}\right)_j}{\left(\max \dot{v}_{\text{ref}} - \min \dot{v}_{\text{ref}}\right)_j}.$$

So E_{dv} contains the $\dot{x}$, $\dot{y}$ and $\dot{z}$ errors, each divided by its own band, and averaged.

Simulation error on validation data. The SINDy equations are solved from the starting point of validation run j, giving $x_{\text{SINDy}}(t), y_{\text{SINDy}}(t), z_{\text{SINDy}}(t)$, and compared with the true $x(t), y(t), z(t)$ over the window $0 \le t \le T$:

$$E_{sim}(T) = \frac{1}{3}\sum_{j=1}^{3} \frac{1}{3} \sum_{v \in \{x,y,z\}} \frac{\text{RMS}_{[0,T]}\left(v_{\text{SINDy}} - v\right)_j}{\left(\max_{[0,T]} v - \min_{[0,T]} v\right)_j}.$$

T is kept short (0.5, 1, 2) because the Lorenz system is chaotic: over a long window every model, even an excellent one, drifts away from the true trajectory. A model whose solution grows without bound gets $E_{sim} = \infty$.

Maximum coefficient error. Lorenz is the only case where the true coefficients are known, so one more number can be given: the largest of the $56 \times 3 = 168$ differences between the SINDy coefficients and the true ones,

$$\text{max coef. err.} = \max_{i,k} \left|\Xi_{ik,\text{SINDy}} - \Xi_{ik,\text{true}}\right|,$$

where the true Ξ has the 7 Lorenz coefficients and 161 zeros. E_{dv} and E_{sim} say whether the model *behaves* correctly; this number says whether its *coefficients* are correct.

Decided: the main measure for Lorenz is E_{dv}: SINDy predicts derivatives, and for Lorenz the target is $\dot{x}, \dot{y}, \dot{z}$. E_{sim} is reported alongside. (For the battery the target is C_s, so there the main measure is E_{sim}; Section 8.3.3.)

8.2.4 Sweeping the threshold

λ is swept from 10^{-12} to $10^{1.5} \approx 32$ (four values per decade). Rows giving the same model are merged. "Wrong": a term not in the true equations. "Missing": a true term that was removed. The shaded row is the model chosen in Section 8.2.7.

λ range	terms	wrong	missing	max coef. err.	E_{train}	E_{dv}	$E_{sim}(0.5)$	$E_{sim}(1)$	$E_{sim}(2)$
1.0×10^{1} – 3.2×10^{1}	0	0	7	2.8×10^{1}	1.6×10^{-1}	1.6×10^{-1}	4.8×10^{-1}	3.9×10^{-1}	3.5×10^{-1}
3.2×10^{0} – 5.6×10^{0}	2	0	5	2.8×10^{1}	1.0×10^{-1}	1.0×10^{-1}	4.1×10^{-1}	3.3×10^{-1}	3.2×10^{-1}
1.8×10^{0}	3	0	4	3.1×10^{1}	1.0×10^{-1}	9.8×10^{-2}	4.5×10^{-1}	3.8×10^{-1}	3.1×10^{-1}
1.0×10^{0}	6	0	1	4.1×10^{1}	2.7×10^{-2}	2.8×10^{-2}	6.3×10^{-1}	6.1×10^{-1}	5.0×10^{-1}
1.0×10^{-12} – 5.6×10^{-1}	7	0	0	5.2×10^{-14}	2.2×10^{-16}	2.2×10^{-16}	4.8×10^{-11}	5.8×10^{-11}	2.0×10^{-10}

Table 6: Case A, exact derivative. Errors as fractions.

λ range	terms	wrong	missing	max coef. err.	E_{train}	E_{dv}	$E_{sim}(0.5)$	$E_{sim}(1)$	$E_{sim}(2)$
1.0×10^{1} – 3.2×10^{1}	0	0	7	2.8×10^{1}	1.6×10^{-1}	1.6×10^{-1}	4.8×10^{-1}	3.9×10^{-1}	3.5×10^{-1}
3.2×10^{0} – 5.6×10^{0}	2	0	5	2.8×10^{1}	1.0×10^{-1}	1.0×10^{-1}	4.1×10^{-1}	3.3×10^{-1}	3.2×10^{-1}
1.0×10^{0} – 1.8×10^{0}	3	0	4	3.1×10^{1}	1.0×10^{-1}	9.8×10^{-2}	4.5×10^{-1}	3.8×10^{-1}	3.1×10^{-1}
5.6×10^{-5} – 5.6×10^{-1}	7	0	0	2.0×10^{-3}	4.8×10^{-6}	4.9×10^{-6}	2.3×10^{-5}	5.4×10^{-5}	1.0×10^{-4}
3.2×10^{-5}	8	1	0	2.0×10^{-3}	4.6×10^{-6}	4.7×10^{-6}	2.1×10^{-5}	6.1×10^{-5}	1.4×10^{-4}
1.8×10^{-5}	9	2	0	2.0×10^{-3}	4.4×10^{-6}	4.5×10^{-6}	2.1×10^{-5}	5.4×10^{-5}	1.3×10^{-4}
1.0×10^{-5}	12	5	0	2.8×10^{-3}	3.8×10^{-6}	3.9×10^{-6}	4.6×10^{-5}	7.1×10^{-5}	1.7×10^{-4}
5.6×10^{-6}	14	7	0	2.8×10^{-3}	2.3×10^{-6}	2.5×10^{-6}	4.7×10^{-5}	5.7×10^{-5}	1.3×10^{-4}
1.8×10^{-6} – 3.2×10^{-6}	17	10	0	1.2×10^{-3}	1.2×10^{-6}	1.4×10^{-6}	1.8×10^{-5}	4.5×10^{-5}	8.6×10^{-5}
1.8×10^{-7} – 1.0×10^{-6}	19	12	0	3.7×10^{-3}	3.3×10^{-7}	3.6×10^{-7}	1.8×10^{-5}	2.7×10^{-5}	3.1×10^{-5}
1.8×10^{-9} – 1.0×10^{-7}	21	14	0	1.8×10^{-3}	1.3×10^{-10}	1.4×10^{-10}	2.3×10^{-5}	3.1×10^{-5}	3.5×10^{-5}
1.0×10^{-9}	22	15	0	1.8×10^{-3}	1.3×10^{-10}	1.3×10^{-10}	2.3×10^{-5}	3.1×10^{-5}	3.5×10^{-5}
5.6×10^{-10}	23	16	0	1.8×10^{-3}	1.3×10^{-10}	1.3×10^{-10}	2.3×10^{-5}	3.1×10^{-5}	3.5×10^{-5}
3.2×10^{-10}	24	17	0	1.8×10^{-3}	1.3×10^{-10}	1.3×10^{-10}	2.3×10^{-5}	3.1×10^{-5}	3.5×10^{-5}
1.0×10^{-10}	32	25	0	1.8×10^{-3}	1.1×10^{-10}	1.2×10^{-10}	2.3×10^{-5}	3.1×10^{-5}	3.5×10^{-5}
5.6×10^{-11}	42	35	0	1.8×10^{-3}	1.0×10^{-10}	1.1×10^{-10}	2.3×10^{-5}	3.1×10^{-5}	3.5×10^{-5}
3.2×10^{-11}	38	31	0	1.8×10^{-3}	8.7×10^{-11}	9.8×10^{-11}	2.3×10^{-5}	3.1×10^{-5}	3.5×10^{-5}
1.8×10^{-11}	46	39	0	1.8×10^{-3}	3.2×10^{-11}	3.9×10^{-11}	2.3×10^{-5}	3.1×10^{-5}	3.5×10^{-5}
1.0×10^{-11}	53	46	0	1.8×10^{-3}	3.0×10^{-11}	3.7×10^{-11}	2.3×10^{-5}	3.1×10^{-5}	3.5×10^{-5}
5.6×10^{-12}	60	53	0	1.8×10^{-3}	2.1×10^{-11}	2.6×10^{-11}	2.3×10^{-5}	3.1×10^{-5}	3.5×10^{-5}
3.2×10^{-12}	62	55	0	1.8×10^{-3}	1.6×10^{-11}	2.0×10^{-11}	2.3×10^{-5}	3.1×10^{-5}	3.5×10^{-5}
1.8×10^{-12}	58	51	0	1.8×10^{-3}	6.5×10^{-12}	9.4×10^{-12}	2.3×10^{-5}	3.1×10^{-5}	3.5×10^{-5}
1.0×10^{-12}	64	57	0	1.8×10^{-3}	6.4×10^{-12}	9.5×10^{-12}	2.3×10^{-5}	3.1×10^{-5}	3.5×10^{-5}

Table 7: Case C2, 2nd-order finite differences.

λ range	terms	wrong	missing	max coef. err.	E_{train}	E_{dv}	$E_{sim}(0.5)$	$E_{sim}(1)$	$E_{sim}(2)$
1.0×10^{1} – 3.2×10^{1}	0	0	7	2.8×10^{1}	1.6×10^{-1}	1.6×10^{-1}	4.8×10^{-1}	3.9×10^{-1}	3.5×10^{-1}
3.2×10^{0} – 5.6×10^{0}	2	0	5	2.8×10^{1}	1.0×10^{-1}	1.0×10^{-1}	4.1×10^{-1}	3.3×10^{-1}	3.2×10^{-1}
1.0×10^{0} – 1.8×10^{0}	3	0	4	3.1×10^{1}	1.0×10^{-1}	9.8×10^{-2}	4.5×10^{-1}	3.8×10^{-1}	3.1×10^{-1}
1.8×10^{-8} – 5.6×10^{-1}	7	0	0	2.3×10^{-7}	1.1×10^{-9}	1.1×10^{-9}	2.7×10^{-9}	5.7×10^{-9}	1.2×10^{-8}
1.0×10^{-8}	9	2	0	8.5×10^{-7}	1.1×10^{-9}	1.1×10^{-9}	2.5×10^{-9}	5.6×10^{-9}	1.7×10^{-8}
5.6×10^{-9}	10	3	0	8.5×10^{-7}	1.1×10^{-9}	1.1×10^{-9}	2.5×10^{-9}	5.5×10^{-9}	1.7×10^{-8}
3.2×10^{-9}	13	6	0	8.5×10^{-7}	9.7×10^{-10}	1.0×10^{-9}	5.1×10^{-9}	6.4×10^{-9}	1.8×10^{-8}
1.8×10^{-9}	18	11	0	8.8×10^{-7}	5.5×10^{-10}	5.9×10^{-10}	4.9×10^{-9}	7.4×10^{-9}	1.2×10^{-8}
1.0×10^{-9}	19	12	0	1.4×10^{-6}	5.5×10^{-10}	5.9×10^{-10}	4.8×10^{-9}	6.9×10^{-9}	1.0×10^{-8}
5.6×10^{-10}	22	15	0	1.4×10^{-6}	6.0×10^{-10}	6.4×10^{-10}	3.7×10^{-9}	6.0×10^{-9}	1.4×10^{-8}
3.2×10^{-10}	32	25	0	9.0×10^{-7}	4.7×10^{-10}	4.9×10^{-10}	2.9×10^{-9}	4.2×10^{-9}	1.5×10^{-8}
1.8×10^{-10}	38	31	0	5.6×10^{-7}	3.6×10^{-10}	3.8×10^{-10}	8.1×10^{-9}	1.4×10^{-8}	1.9×10^{-8}
1.0×10^{-10}	46	39	0	9.8×10^{-8}	2.3×10^{-10}	2.5×10^{-10}	4.2×10^{-9}	5.5×10^{-9}	9.2×10^{-9}
5.6×10^{-11}	51	44	0	7.9×10^{-7}	1.3×10^{-10}	1.6×10^{-10}	2.2×10^{-9}	3.7×10^{-9}	7.3×10^{-9}
3.2×10^{-11}	56	49	0	7.4×10^{-7}	1.2×10^{-10}	1.5×10^{-10}	2.1×10^{-9}	2.6×10^{-9}	4.0×10^{-9}
1.8×10^{-11}	60	53	0	7.7×10^{-7}	8.4×10^{-11}	1.0×10^{-10}	3.1×10^{-9}	3.3×10^{-9}	5.5×10^{-9}
1.0×10^{-11}	62	55	0	2.1×10^{-6}	3.9×10^{-11}	5.4×10^{-11}	1.8×10^{-9}	2.7×10^{-9}	4.7×10^{-9}
5.6×10^{-12}	64	57	0	2.9×10^{-6}	2.4×10^{-11}	3.6×10^{-11}	1.4×10^{-9}	2.0×10^{-9}	4.0×10^{-9}
3.2×10^{-12}	53	46	0	1.2×10^{-7}	6.8×10^{-12}	9.4×10^{-12}	2.6×10^{-9}	2.9×10^{-9}	6.1×10^{-9}
1.8×10^{-12}	58	51	0	1.3×10^{-7}	5.8×10^{-12}	7.7×10^{-12}	2.3×10^{-9}	2.4×10^{-9}	5.0×10^{-9}
1.0×10^{-12}	58	51	0	1.3×10^{-7}	5.8×10^{-12}	7.7×10^{-12}	2.3×10^{-9}	2.4×10^{-9}	5.0×10^{-9}

Table 8: Case C4, 4th-order finite differences.

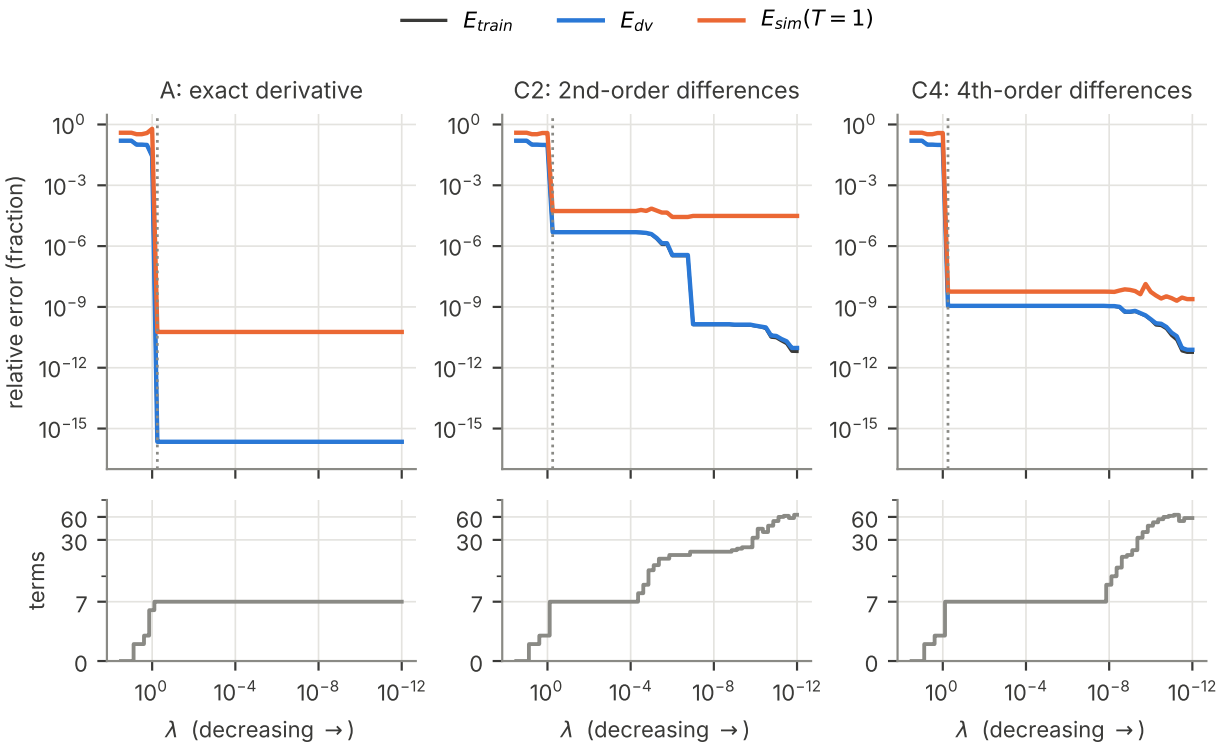


Figure 37: The three error measures against λ (λ decreases to the right); E_{train} lies under E_{dv} almost everywhere. Lower panels: number of terms. Dotted line: the selected $\lambda = 0.56$.

What the sweep shows:

- Going down from large λ, the model gains terms one group at a time. Between $\lambda = 1$ and 0.56 it reaches the 7 true terms, and every error drops by roughly 4 to 14 orders of magnitude. This is the largest change in the sweep.
- Below that, the errors stay flat over a wide plateau. Further down, wrong terms appear: below 5.6×10^{-5} for C2 and below 1.8×10^{-8} for C4. With the exact derivative (A) none appear even at 10^{-12}.
- For C2, E_{train} and E_{dv} fall a *second* time, by about 3.4 orders, when 21 terms are kept ($\lambda = 10^{-7}$). The error of the 2nd-order difference is $\frac{\Delta t^2}{6}\dddot{x}$, and for Lorenz $\dddot{x}$, $\dddot{y}$, $\dddot{z}$ are themselves polynomials in x, y, z of degree at most 4. They lie inside the fifth-order library, so the extra (wrong) terms can fit the finite-difference error almost exactly. E_{dv} is fooled too, because the validation derivative is taken with the same formula and has the same error. E_{sim} hardly moves (from 5.4×10^{-5} to 3.1×10^{-5}), and the coefficient error does not improve. A smaller derivative error is therefore not always a better model.

8.2.5 Finding a rule for choosing λ

The sweep gives many models. The professor asked how anyone given such a sweep should pick one, in a way that works for other problems too. Four rules were tried, each on E_{dv} and on $E_{sim}(T{=}1)$. Each cell gives the number of terms of the chosen model and, in brackets, how many of them are wrong; the correct answer is 7 (0).

Case	Error	R1	R2	R3	R4
A	E_{dv}	7 (0)	7 (0)	7 (0)	7 (0)
A	$E_{sim}(T{=}1)$	7 (0)	7 (0)	7 (0)	7 (0)
C2	E_{dv}	58 (51)	46 (39)	7 (0)	7 (0)
C2	$E_{sim}(T{=}1)$	19 (12)	7 (0)	7 (0)	7 (0)
C4	E_{dv}	58 (51)	62 (55)	7 (0)	7 (0)
C4	$E_{sim}(T{=}1)$	64 (57)	7 (0)	7 (0)	7 (0)

Table 9: Terms (wrong terms) of the model picked by each rule.

1. **R1: the largest λ whose error is within 10% of the smallest error.** The natural first idea: take the most accurate model. It *failed* whenever the derivative was approximate, choosing models with up to 57 wrong terms. Lesson: the smallest error belongs to an overfitted model, because the extra terms absorb the error of the finite-difference derivative.
2. **R2: within a factor 10 of the smallest error.** A looser tolerance. It still *failed* with E_{dv} for C2 and C4. Lesson: any rule anchored to the minimum error inherits its overfitting; how much tolerance is enough depends on the case.
3. **R3: the largest λ whose error is below a fixed level (10^{-3}).** It *worked* in every case. Its weakness: the level 10^{-3} was chosen by looking at these results, and a different problem, with different units and accuracy, would need a different level.
4. **R4: the model just after the largest drop in error (the "knee").** It *worked* in every case and needs no level to be chosen: it only looks for where adding terms stops paying off. It is also unaffected by any constant scaling of the error, since it compares drops in $\log_{10}$ of the error.

Why: R4 was chosen. It found the correct model in all six combinations, like R3, but it needs no problem-specific number, so it can be carried over to the PDE and to other systems unchanged.

A warning from C2. With E_{dv}, the largest drop ($3 \to 7$ terms) is 4.3 orders of magnitude, but the second largest ($19 \to 21$ terms, the finite-difference fit described above) is 3.4 orders. R4 picks the right model, but by a smaller margin than for A and C4 (14.1 against 0.6, and 7.9 against 0.6 orders).

8.2.6 The rule, step by step

1. **Sweep.** Run STLSQ for λ on a log scale (four values per decade), from a value large enough to remove every term down to a value where unnecessary terms appear.
2. **Merge.** λ values that give the same model are merged into one model with a λ range.
3. **Score.** Compute each model's error on validation data that were not used to fit it (E_{dv} for Lorenz, E_{sim} for the PDE).
4. **Discard unusable models.** Remove models whose solution grows without bound when they are solved (unstable models). The empty model (all coefficients zero) is kept. *This step was added after a failure on the PDE, Section 8.3.5.*
5. **Order.** Arrange the remaining models from largest to smallest λ.
6. **Find the knee.** Between each pair of neighbours compute the drop in error, $\log_{10} E_{\text{before}} - \log_{10} E_{\text{after}}$, and find the largest drop.
7. **Select.** Take the model just after the largest drop.
8. **Choose λ.** Use the largest λ of that model's range: it gives the same model with the fewest small coefficients to resolve, so it is the cheapest and least sensitive choice.
9. **Test.** Score the selected model once on the test data. Nothing is changed after this step.

8.2.7 Applying the rule to Lorenz

Case	Score	terms	wrong	λ range of the selected model	selected λ
A	E_{dv} (main)	7	0	1.0×10^{-12} – 5.6×10^{-1}	5.62×10^{-1}
A	$E_{sim}(T{=}1)$	7	0	1.0×10^{-12} – 5.6×10^{-1}	5.62×10^{-1}
C2	E_{dv} (main)	7	0	5.6×10^{-5} – 5.6×10^{-1}	5.62×10^{-1}
C2	$E_{sim}(T{=}1)$	7	0	5.6×10^{-5} – 5.6×10^{-1}	5.62×10^{-1}
C4	E_{dv} (main)	7	0	1.8×10^{-8} – 5.6×10^{-1}	5.62×10^{-1}
C4	$E_{sim}(T{=}1)$	7	0	1.8×10^{-8} – 5.6×10^{-1}	5.62×10^{-1}

Table 10: The rule (R4) applied to every Lorenz case.

In every case and with both error measures, the rule selects the true 7-term model and $\lambda = 0.56$. The selected equations are those of Table 5.

8.2.8 Test: the selected model on three unseen runs

The selected model is scored once on the three test runs, with the same formulas as E_{dv} and E_{sim}. For comparison, the true Lorenz equations are scored on the same runs in the same way.

Case	Model	E_{dv}	$E_{sim}(0.5)$	$E_{sim}(1)$	$E_{sim}(2)$
A	SINDy, selected (7 terms)	2.2×10^{-16}	4.7×10^{-11}	3.6×10^{-11}	9.4×10^{-11}
A	true Lorenz equations	2.0×10^{-17}	4.7×10^{-11}	3.6×10^{-11}	9.4×10^{-11}
C2	SINDy, selected (7 terms)	4.9×10^{-6}	3.7×10^{-5}	4.5×10^{-5}	1.4×10^{-4}
C2	true Lorenz equations	6.8×10^{-6}	4.7×10^{-11}	3.6×10^{-11}	9.4×10^{-11}
C4	SINDy, selected (7 terms)	1.1×10^{-9}	4.7×10^{-9}	4.8×10^{-9}	1.6×10^{-8}
C4	true Lorenz equations	1.2×10^{-9}	4.7×10^{-11}	3.6×10^{-11}	9.4×10^{-11}

Table 11: Test errors (fractions), mean over the three test runs.

- The test errors are of the same size as the validation errors of the same model: nothing was tuned to the validation runs.
- For C2 and C4 the SINDy model has a *smaller* E_{dv} than the true equations. This is not a better model: the reference derivative is itself a finite difference, and SINDy has partly fitted its error. E_{sim}, which compares states and not derivatives, puts the true equations ahead.
- The true equations still have $E_{sim} \approx 4\times10^{-11}$. This is the tolerance of the ODE solver (10^{-10}), the floor of this measure.

8.3 SINDy on the PDE, one frequency at a time

Why: the Galerkin model has fixed coefficients, the same at every frequency. An earlier attempt stacked 20 sine runs from $\omega = 0.5$ to 200 into one fit and found $dC_s/d\tau = -52.88\,(C_s - C_{\text{avg}}) - 12.62\,\delta$; the fast runs, whose $dC_s/d\tau$ is much larger, dominated the least-squares fit, so a single answer hid how the coefficients depend on frequency. Fitting SINDy separately at each frequency shows whether the data agree with the Galerkin coefficients, and where they stop agreeing.

8.3.1 Training runs

ω	period P	$\Delta\tau = P/200$	$\tau_{\max} = 5P$	points
0.1	62.83	0.3142	314.2	1001
1	6.283	0.03142	31.42	1001
10	0.6283	0.003142	3.142	1001
100	0.06283	0.0003142	0.3142	1001

Table 12: Training runs: $\delta = A\sin(\omega\tau)$, $f = 1$.

- Library $\{C_s,\ C_{\text{avg}},\ \delta\}$, the terms of the Galerkin model, so that the coefficients can be compared one to one. SINDy fits only $dC_s/d\tau$; C_{avg} always follows the exact mass balance, Equation (7).
- $dC_s/d\tau$ by 2nd-order differences at all 1001 points: central differences inside, and one-sided 2nd-order differences at the first and last point.
- The PDE data for this section come from a Python solver (Chebyshev collocation in r with an implicit time integrator, tolerance 10^{-10}; doubling the radial points changes C_s by less than 10^{-9}). It reproduces the Galerkin errors of Table 4 to within about 1%.

Why: 200 points per period and 5 periods give every frequency the same number of points (1001) and the same resolution of each oscillation, so no frequency is favoured or under-resolved.

8.3.2 Validation and test data

Each model is scored on four validation runs at the same ω that were not used in the fit, each **10 periods** long with $\Delta\tau = P/200$:

1. **cos**: $\delta = A\cos(\omega\tau)$. The flux starts at full size, so it also tests the response to a sudden start.
2. **sin late**: periods 6 to 15 of the training sine (the PDE is continued from its state at $\tau = 5P$, and the model starts from the PDE's C_s and C_{avg} there). Same signal, later time.
3. **sin ph**: $\delta = A\sin(\omega\tau + \pi/4)$.
4. **cos ph**: $\delta = A\cos(\omega\tau + \pi/4)$.

"Stacked" is the mean of the four errors. **Decided:** validation runs extended from 5 to 10 periods; all four kept and stacked.

Test (used once): the random multisine $\delta = s\sum_{k=1}^{3} a_k \sin(\omega_k\tau + \phi_k)$ with $\omega_k = \omega/2,\ \omega,\ 2\omega$, a_k and ϕ_k as in Table 13, scaled so that $\max|\delta| = A$, 10 periods of ω. **Decided:** the random input is removed from validation and becomes the test set. *Why:* the cos, sin late and phase-shifted runs all test the frequency the model was trained at. The random input also contains $\omega/2$ and 2ω, which the model never saw, and it is the input closest to a real battery current. That makes it a harder, more honest final check rather than a way of choosing λ.

ω	ω_k	a_k	ϕ_k
0.1	0.05, 0.1, 0.2	0.82, 0.74, 0.75	4.62, 3.09, 0.48
1	0.5, 1, 2	0.54, 0.91, 0.82	1.24, 3.70, 5.63
10	5, 10, 20	0.91, 0.53, 0.69	4.99, 2.70, 4.75
100	50, 100, 200	0.62, 0.97, 0.53	1.31, 0.81, 2.65

Table 13: Parameters of the random (test) inputs.

8.3.3 How the error of a model is measured

Equation (25) with a single variable, in %.

- E_{sim}: the SINDy equation for C_s is solved together with $dC_{\text{avg}}/d\tau = -3\delta$, from the PDE's C_s and C_{avg} at the start of the run, and compared with the PDE's C_s over the **whole run**:

$$E_{sim}\,(\%) = 100 \times \frac{\text{RMS}\,(C_{s,\text{SINDy}} - C_{s,\text{PDE}})}{\max C_{s,\text{PDE}} - \min C_{s,\text{PDE}}} \quad \text{(whole run)}.$$

  Only C_s is compared: every model gets C_{avg} exactly, from the mass balance.

- E_{dv}: the SINDy right-hand side, evaluated at the PDE's C_s, C_{avg} and δ, against $dC_s/d\tau$ of the PDE data (2nd-order differences, as in training), whole run:

$$E_{dv}\,(\%) = 100 \times \frac{\text{RMS}\,(\xi_1 C_s + \xi_2 C_{\text{avg}} + \xi_3 \delta - dC_s/d\tau)}{\max dC_s/d\tau - \min dC_s/d\tau}.$$

- E_{train}: the same as E_{dv} on the training run.

Decided: for the battery the main measure is E_{sim} on C_s, over the whole run. The aim of the model is C_s: even if $dC_s/d\tau$ is predicted imperfectly, C_s must come out right.

8.3.4 Models found

		coefficients				E_{sim} (%) on validation runs					derivative error (%)	
ω	λ range	C_s	C_{avg}	δ	stable	cos	sin late	sin ph	cos ph	stacked	E_{dv}, stacked	E_{train}
0.1	1.8×10^{1} – 1.0×10^{3}	+0.000	+0.000	+0.000	yes	35	61	50	50	49	34.2	35.3
0.1	1.0×10^{1}	+14.998	-14.999	+0.000	no	∞	∞	∞	∞	∞	0.961	0.144
0.1	1.0×10^{-6} – 5.6×10^{0}	-34.638	+34.638	-9.927	yes	2.3×10^{-5}	1.7×10^{-5}	1.9×10^{-5}	2.7×10^{-5}	2.1×10^{-5}	1.26	9.06×10^{-3}
1	1.8×10^{1} – 1.0×10^{3}	+0.000	+0.000	+0.000	yes	35	61	50	50	49	24.4	35.3
1	1.0×10^{-6} – 1.0×10^{1}	-35.039	+35.039	-10.004	yes	5.4×10^{-3}	1.8×10^{-4}	3.8×10^{-3}	3.9×10^{-3}	3.3×10^{-3}	0.136	0.0436
10	1.8×10^{1} – 1.0×10^{3}	+0.000	+0.000	+0.000	yes	35	57	44	45	45	17.4	35.2
10	1.0×10^{-6} – 1.0×10^{1}	-38.077	+38.077	-10.313	yes	0.25	0.042	0.19	0.19	0.17	0.999	0.641
100	3.2×10^{1} – 1.0×10^{3}	+0.000	+0.000	+0.000	yes	35	52	35	35	39	16.8	34.5
100	1.8×10^{1}	+28.745	-28.738	+0.000	no	∞	∞	∞	∞	∞	16.4	33.5
100	1.0×10^{-6} – 1.0×10^{1}	-133.216	+133.207	-17.630	yes	1.7	1.4	1.5	2.1	1.7	1.86	2.82

Table 14: Models found in the λ sweep (10^{-6} to 10^{3}, four values per decade) and their errors in %. Each row is one candidate model; each validation column is that model's E_{sim} on that run. ∞: the solution grows without bound. The empty model keeps C_s at its starting value. Shaded: the selected model.

8.3.5 Choosing λ with the rule

	E_{sim} on					E_{dv}, stacked			
ω	cos	sin late	sin ph	cos ph	**stacked**	step 4 skipped	step 4 used	λ range	selected λ
0.1	3	3	3	3	3	2, **unstable**	3	1.0×10^{-6} – 5.6×10^{0}	5.6×10^{0}
1	3	3	3	3	3	3	3	1.0×10^{-6} – 1.0×10^{1}	1.0×10^{1}
10	3	3	3	3	3	3	3	1.0×10^{-6} – 1.0×10^{1}	1.0×10^{1}
100	3	3	3	3	3	3	3	1.0×10^{-6} – 1.0×10^{1}	1.0×10^{1}

Table 15: Number of terms of the model selected by R4 for every choice of validation error. The selection used is E_{sim}, stacked.

- With E_{sim}, every validation run and the stacked set select the same three-term model at every frequency.
- **A failure that changed the rule.** The rule was first applied without step 4. With the derivative error at $\omega = 0.1$ it then chose the two-term model, because that model fits the derivatives slightly better (0.96% against 1.26% stacked E_{dv}). But its C_s coefficient is positive, so when it is solved C_s grows without bound. A model that fits derivatives can still be useless for prediction. Step 4, removing unstable models before looking for the knee, was added because of this, and with it the derivative error also selects the three-term model.

- With only three library terms, λ has little to choose: below about 5–10 every λ gives the same model. The choice of validation data will matter more once the library is larger.
- The selected λ is 5.6 at $\omega = 0.1$ and 10 at the other frequencies (the largest values on the sweep grid that still give the three-term model).

8.3.6 Test on the random input

The selected model at each ω is scored once on the random multisine. The Galerkin model is scored on the same runs.

ω	E_{sim} (%) Galerkin	E_{sim} (%) SINDy	E_{dv} (%) Galerkin	E_{dv} (%) SINDy	E_{sim}: Galerkin / SINDy
0.1	2.02×10^{-6}	1.81×10^{-5}	0.583	0.576	0.11
1	2.18×10^{-3}	1.79×10^{-3}	0.143	0.13	1.2
10	0.756	0.563	1.61	1.46	1.3
100	11.5	3.93	4.03	2.38	2.9

Table 16: Test errors on the random multisine (10 periods, whole run), in %.

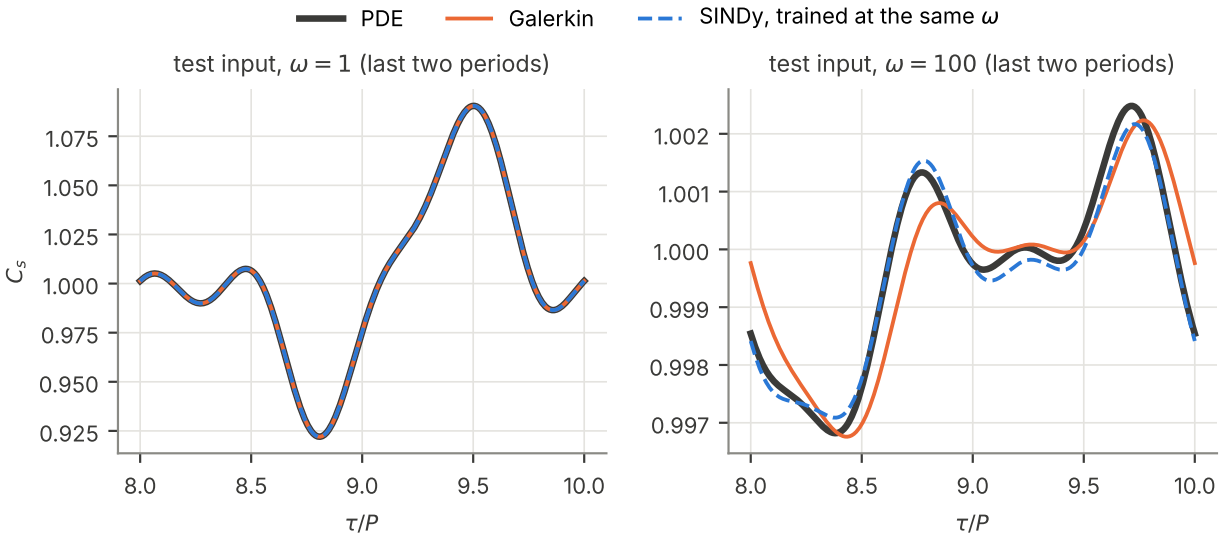


Figure 38: C_s on the test input over its last two periods, at $\omega = 1$ (all three curves coincide) and at $\omega = 100$.

- At $\omega = 1$ and 10 SINDy and Galerkin are close (SINDy 1.2–1.3 times better). At $\omega = 100$ the SINDy model is about 3 times more accurate than Galerkin (3.9% against 11.5%), even though the test input contains $\omega/2$ and 2ω, which it never saw.
- At $\omega = 0.1$ Galerkin is better (2×10^{-6}% against 2×10^{-5}%), both being tiny. The reason is the 1% offset of the $\omega = 0.1$ coefficients, explained in the next section.

8.3.7 Selected equations against the Galerkin model

	derivative	C_s	C_{avg}	δ
Galerkin	–	-35	$+35$	-10
$\omega = 0.1$	2nd order	-34.638	+34.638	-9.927
	4th order	-34.665	+34.665	-9.933
	2nd order, end points dropped	-34.996	+34.996	-9.999
$\omega = 1$	2nd order	-35.039	+35.039	-10.004
	4th order	-35.048	+35.048	-10.006
	2nd order, end points dropped	-35.040	+35.040	-10.004
$\omega = 10$	2nd order	-38.077	+38.077	-10.313
	4th order	-38.084	+38.084	-10.315
	2nd order, end points dropped	-38.078	+38.078	-10.313
$\omega = 100$	2nd order	-133.216	+133.207	-17.630
	4th order	-133.239	+133.229	-17.633
	2nd order, end points dropped	-133.225	+133.215	-17.630

Table 17: Coefficients of the selected model at each training frequency. "End points dropped": the same 2nd-order fit without the first and last point.

- $\omega = 1$: -35.04, -10.004, the Galerkin model to within 0.1%, as expected for a slow flux.
- $\omega = 0.1$: -34.6, -9.93, about 1% off. Here $\Delta\tau = P/200 = 0.31$ is coarser than the start-up, which is over by about $\tau = 0.5$, so the derivative at the first point (a one-sided difference across the start-up) is poor. **Check:** leaving out just the first and last point gives -34.996, -9.999, the Galerkin values. So the offset is a sampling effect, not physics.
- $\omega = 10$: -38.1, -10.3. $\omega = 100$: -133, -17.6, far from Galerkin. Leaving out the end points changes nothing here.
- In every case the C_s and C_{avg} coefficients are equal and opposite, as in the Galerkin model. SINDy was not told this; it comes out of the data.
- 4th-order differences change the coefficients only in the 3rd or 4th digit.

8.3.8 Cross-test: trained at one frequency, tested at another

Every selected model is tested on the cosine run at each ω, over the last period, with two error measures:

- C_s: Rel (%) of C_s, Equation (25), last period.
- gap: the same RMS error divided by the band of the gap $C_s - C_{\text{avg}}$ instead. Every model gets C_{avg} exactly (mass balance), so the only thing a model can get wrong is the gap. At low ω the swing of C_s is almost all C_{avg}, which hides gap errors in the C_s measure.

Decided: keep both measures.

Model	test $\omega = 0.1$		test $\omega = 1$		test $\omega = 10$		test $\omega = 100$	
	C_s	gap	C_s	gap	C_s	gap	C_s	gap
Galerkin	1.52×10^{-6}	1.61×10^{-4}	1.07×10^{-3}	0.0161	0.704	1.48	16.3	20.1
SINDy, trained at $\omega = 0.1$	2.18×10^{-5}	3.17×10^{-3}	1.15×10^{-3}	0.0174	0.715	1.51	16.4	20.3
SINDy, trained at $\omega = 1$	1.23×10^{-4}	0.0184	1.85×10^{-4}	2.72×10^{-3}	0.695	1.46	16.3	20.1
SINDy, trained at $\omega = 10$	9.40×10^{-3}	1.41	0.0925	1.39	0.0423	0.089	15.6	19.3
SINDy, trained at $\omega = 100$	0.107	16	1.06	16	7.29	15.4	1.43	1.77

Table 18: Cross-test, Rel (%), cosine flux, last period.

- Each model is best at the frequency it was trained on.
- The models trained at $\omega = 0.1$ and 1 behave like the Galerkin model: excellent at low ω, about 20% gap error at $\omega = 100$.
- The model trained at $\omega = 100$ is good at $\omega = 100$ (1.8% gap error) but has about 16% gap error at lower frequencies. On the C_s measure the same model looks fine at $\omega = 0.1$ (0.1%), which is why the gap measure is kept as well.
- No single set of coefficients for $\{C_s, C_{\text{avg}}, \delta\}$ works at both low and high frequency. This is the reason for adding new terms to the library.

9 Summary

- Changing the flux shape changes the surface concentration response. The average concentration is set by the total charge applied, so matched pairs with the same total charge end at the same C_{avg} but different C_s.
- Increasing the frequency of an oscillating flux reduces the visible oscillation in C_s and confines the response to a thinner layer near the surface, with the material model unchanged.
- Increasing the constant diffusivity makes the radial profile more uniform. The maximum gap $C_{\text{avg}} - C_s$ falls from 0.0710 at $f = 0.01$ to 0.000051 at $f = 100$.
- The average concentration follows the applied flux and is essentially independent of the diffusivity, as Equation (7) requires.
- `trueRandom` gives different reproducible excitation signals for different seeds, with surface responses that are similar in character but different in detail.
- The Galerkin method with the two shapes 1 and $r^2 - 3/5$ turns the PDE into two ODEs: $dC_{\text{avg}}/d\tau = -3\delta$ and $dC_s/d\tau = -35 f(C_s - C_{\text{avg}}) - 10\,\delta$. Measured by the relative error over the last period, it agrees with the PDE to 0.0001% at $\omega = 0.5$ and loses accuracy quickly above $\omega \approx 10$, reaching 21% at $\omega = 200$.
- One error formula is used throughout: the RMS error divided by the band (maximum minus minimum) of the reference over the same window, per variable, then averaged.
- On the Lorenz system, the knee rule R4 applied to E_{dv} finds the true 7-term equations with every derivative (exact, 2nd-order, 4th-order) at $\lambda = 0.56$, and its errors on three unseen test runs match the validation errors. Rules anchored to the smallest error overfit, because extra library terms can fit the finite-difference error.
- On the PDE, with the library $\{C_s, C_{\text{avg}}, \delta\}$ and R4 on E_{sim}, SINDy selects the Galerkin-form model at every frequency, with equal and opposite C_s and C_{avg} coefficients. The coefficients match Galerkin at $\omega \leq 1$ and grow with frequency (-133, $+133$, -17.6 at $\omega = 100$). On the unseen random input the model trained at $\omega = 100$ is about 3 times more accurate than Galerkin. But no single set of coefficients works at all frequencies, so the library must be extended.

10 Choices settled, and the next step

1. **Error formula:** Rel = RMS(error)/(max − min of the reference), both over the same window; for several variables, per variable, then averaged.
2. **Main error measure for Lorenz:** E_{dv} (R4 applied to E_{dv}); $E_{sim}(T)$ reported alongside.
3. **Main error measure for the PDE:** E_{sim} of C_s, window = whole run.
4. **PDE validation data:** cos, sin late, sin ph and cos ph, 10 periods each, stacked.

5. **Test sets:** Lorenz, three more new trajectories; PDE, the random multisine. Each used once, after λ is chosen.
6. λ **grid:** four values per decade.
7. **Cross-test error measure:** both C_s and gap.

Next step (open): library expansion, adding weighted averages $C_w = 3\int_0^1 C(r,\tau)\, r^2 w(r)\, dr$ to the library. Which weight functions $w(r)$ to use is to be understood first (and asked to the TA), then implemented.

A Code for Experiment 6

- **Python** (produces every number and figure of Section 8): `pde.py` (PDE solver and Galerkin model), `common.py` (error formula, STLSQ, rules R1–R4), `lorenz.py`, `pde_sindy.py`, `figures.py`, `tables.py`.
- **MATLAB** (the same method; tested in GNU Octave 8.4): `relErr.m` (error formula), `stlsq.m`, `lambdaGrid.m`, `sweepLambda.m`, `selectLambdaR4.m` (rule R4 with step 4), `solvePDE.m` (PDE solver wrapper), `galerkinModel.m` (Galerkin or SINDy C_s model with the mass balance), `run_lorenz.m`, `run_pde_omega.m` (one training frequency), `run_pde_all.m` (all frequencies and the cross-test), `test_quick.m` (checks against the Python results).
- `solvePDE.m` uses `pdepe` when it is available (MATLAB) and otherwise the same Chebyshev method as the Python solver (Octave has no `pdepe`). The `pdepe` branch has not been run yet, since only Octave was available.